University of Pisa
Master's Degree in Computer Science,
Artificial Intelligence

Computational Mathematics for Learning & Data Analysis (646AA)

Group number: 63
Project number: 25 ML

Members:
Matthew Bernardi
Gabriele Marsili

Academic Year: 2025/2026

Contents

Chapter 1

Introduction to the problem

1.1 Problem 25 ML

25. (P) is so-called *extreme learning*, i.e., a neural network with one hidden layer, $y = w\sigma(W_1 x)$, where the weight matrix for the hidden layer W_1 is a fixed random matrix, $\sigma(\cdot)$ is an elementwise activation function of your choice, and the output weight vector w is chosen by solving a least-squares problem with L_1 regularization $\arg\min_w f(w)$, with $f(w) = \frac{1}{2}\|Xw - y\|_2^2 + \lambda\|w\|_1$.

(A1) is *iteratively reweighted least squares*: i.e., an iteration where you solve at each step the least-squares problem

$$w_{k+1} = \arg\min_w f_k(w), \quad f_k(w) = \frac{1}{2}\|Xw - y\|_2^2 + \lambda\|W_k w\|_2^2$$

where W_k is the diagonal matrix with entries $(W_k)_{ii} = |(w_k)_i|^{-1/2}$.

(A2) is an algorithm of the class of the *deflected subgradient methods*.

1.2 Extreme Learning Machine

Extreme Learning Machine (ELM) [22] is a machine learning model implemented as a neural network with a single hidden layer, in which:

- $\mathbf{W_1} \in \mathbb{R}^{H \times N}$ is the input-to-hidden weight matrix (where H is the number of nodes in the hidden layer and N is the number of input nodes) that is **randomly generated** and **static** (does not change during training) [22].
- $\sigma(\cdot)$ is the element-wise (nonlinear) activation function applied to the $\mathbf{W_1 x} \in \mathbb{R}^H$ vector, where $\mathbf{x} \in \mathbb{R}^N$ is the input vector. The choice of using an activation function for the hidden layer results, under mild assumptions and with high probability, in the hidden layer feature matrix with full rank when $M \geq H$, since random projections through a nonlinear function produce linearly independent features.

- $\mathbf{w} \in \mathbb{R}^H$ is the hidden-to-output weight vector, chosen by solving the least-squares problem.
- $\hat{y} = \mathbf{w}^T \sigma(\mathbf{W_1 x}) \in \mathbb{R}$ is the prediction of the model.

The key insight behind this architecture is that randomly fixing $\mathbf{W_1}$ transforms an extremely hard non-linear optimization problem (training a full neural network) into a simpler **linear system** resolution. Classical neural networks must learn *all* weights jointly, which requires non-convex optimization with no guarantee of finding the global minimum [20]. In an ELM, the only free parameters are the output weights $\mathbf{w}$, and once $\mathbf{W_1}$ is fixed, the hidden-layer activations $\mathbf{X} = \sigma(\mathbf{X}_{\text{origin}} \mathbf{W}_1^T)$ are constant. Finding $\mathbf{w}$ then reduces to solving the Least-Squares problem, which is convex with a closed-form solution. This choice works very well in practice: random projections through a non-linear activation function are sufficient to produce rich, expressive features [24] from which a linear model can learn effectively, exploiting the same properties of expressiveness as the **linear basis expansions** [21] in classical Machine Learning models (e.g. linear models).

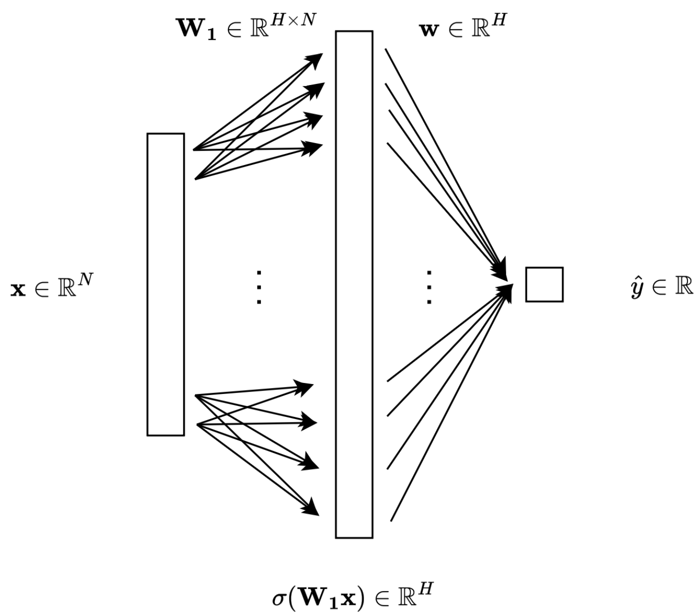

Figure 1.1: Visual representation of an Extreme Learning Machine model.

1.3 Least-Squares Problem

Considering $\mathbf{X} = \sigma(\mathbf{X}_{\text{origin}} \mathbf{W}_1^T) \in \mathbb{R}^{M \times H}$ the matrix of activations of the hidden layer, while $\mathbf{X}_{\text{origin}} \in \mathbb{R}^{M \times N}$ is the matrix of all the inputs, and $\mathbf{y} \in \mathbb{R}^M$ is the vector of all the target values, the definition of the Least Squares Problem [4] in the ELM case is the following:

$$\min_{\mathbf{w}} ||\mathbf{X}\mathbf{w} - \mathbf{y}||_2^2 \tag{1.1}$$

The Least-Squares problem is proven to have a unique solution if and only if the matrix A has full column rank. In that case, we can write the optimization problem as follows:

$$\min_{\mathbf{w}} \frac{1}{2}\mathbf{w}^T\mathbf{X}^T\mathbf{X}\mathbf{w} - \mathbf{y}^T\mathbf{X}\mathbf{w} + \frac{1}{2}\mathbf{y}^T\mathbf{y} \tag{1.2}$$

since the gradient of this function is $\mathbf{X}^T\mathbf{X}\mathbf{w} - \mathbf{X}^T\mathbf{y}$, the Hessian matrix is $\mathbf{X}^T\mathbf{X} \succ 0$, which means that the function is strictly convex. In this case, the minimum exists and is unique, and can be found solving the equation:

$$\mathbf{X}^T\mathbf{X}\mathbf{w} - \mathbf{X}^T\mathbf{y} = 0$$

or

$$\mathbf{X}^T\mathbf{X}\mathbf{w} = \mathbf{X}^T\mathbf{y}$$

where $\mathbf{X}^T\mathbf{X}$ is square invertible since it's positive definite. The linear system has a unique solution $\mathbf{w}_0 = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$. However, adding the L_1 regularization term (Section 1.4) destroys the closed-form structure, requiring iterative methods.

1.4 Regularization Techniques

1.4.1 L_1 regularization

The L_1 regularization [13] is the best convex approximation for the L_0 norm, which penalizes the number of non-zero weights. Because of this sparsity, we are going to use the L_1 instead of L_2, which shrinks the weights but rarely zeroes them out.

When adding the L_1 regularization term to the Least-Squares problem in the case of ELM, the equation is the following:

$$\min_{\mathbf{w}} f(\mathbf{w}) = \frac{1}{2}||\mathbf{X}\mathbf{w} - \mathbf{y}||_2^2 + \lambda_{\text{LASSO}} \sum_{i}^{H} |w_i| \tag{1.3}$$

The price to pay is the non-differentiability of $f(\mathbf{w})$ at any point where some $w_i = 0$, which prevents direct use of gradient-based methods.

Because of that, we can use the Deflected Subgradient Methods (2) and the IRLS algorithm (2), which we will see in the following chapters.

1.5 Objective Function Analysis

The full objective function we need to minimize is the Lasso regularization of the Least-Squares problem:

$$\min_{\mathbf{w}} f(\mathbf{w}) = \underbrace{\frac{1}{2}\|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2}_{g(\mathbf{w})} + \underbrace{\lambda_{\text{LASSO}}\|\mathbf{w}\|_1}_{h(\mathbf{w})}$$

It is useful to analyse the mathematical properties of $f(\mathbf{w})$, as they determine which algorithms can be applied.

1.5.1 Convexity

$f(\mathbf{w})$ is the sum of two convex functions [1]. The quadratic term $g(\mathbf{w}) = \frac{1}{2}\|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2$ has Hessian $\nabla^2 g(\mathbf{w}) = \mathbf{X}^T\mathbf{X} \succeq 0$, making it convex. If $\mathbf{X}$ has full column rank, then $\mathbf{X}^T\mathbf{X} \succ 0$ and g is *strictly convex*. The regularization term $h(\mathbf{w}) = \lambda_{\text{LASSO}}\|\mathbf{w}\|_1$ is convex as a scaled norm. Therefore $f = g + h$ is convex; and if $\mathbf{X}$ has full column rank, f is **strictly convex** and admits a **unique global minimum**.

Proposition 1.1. *If $\mathbf{X} \in \mathbb{R}^{M\times H}$ has full column rank, then $f(\mathbf{w}) = \frac{1}{2}\|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \lambda_{LASSO}\|\mathbf{w}\|_1$ is strictly convex and admits a unique global minimizer $\mathbf{w}^*$ [1].*

Proof. We write $f = g+h$ where $g(\mathbf{w}) = \frac{1}{2}\|\mathbf{X}\mathbf{w}-\mathbf{y}\|_2^2$ and $h(\mathbf{w}) = \lambda_{\text{LASSO}}\|\mathbf{w}\|_1$.

Strict convexity. Since $\mathbf{X}$ has full column rank, for any $\mathbf{v} \neq \mathbf{0}$ we have $\mathbf{X}\mathbf{v} \neq \mathbf{0}$, so $\mathbf{v}^T(\mathbf{X}^T\mathbf{X})\mathbf{v} = \|\mathbf{X}\mathbf{v}\|_2^2 > 0$. Hence the Hessian of g is $\nabla^2 g = \mathbf{X}^T\mathbf{X} \succ 0$, making g strictly convex. h is convex as a non-negative multiple of a norm. The sum of a strictly convex function and a convex function is strictly convex [8], so f is strictly convex.

Existence and uniqueness of the minimizer. Since $\mathbf{X}^T\mathbf{X} \succ 0$, g is coercive: $g(\mathbf{w}) \to +\infty$ as $\|\mathbf{w}\| \to \infty$. Because $h \geq 0$, $f = g + h$ is also coercive. This means the sub-level set $\{\mathbf{w} : f(\mathbf{w}) \leq f(\mathbf{0})\}$ is compact, and since f is continuous, it attains its minimum on this set. Strict convexity then rules out two distinct minimizers: if $\mathbf{w}_1 \neq \mathbf{w}_2$ both minimized f, the midpoint $\frac{\mathbf{w}_1+\mathbf{w}_2}{2}$ would give a strictly smaller value, a contradiction. □

1.5.2 Non-smoothness

Unlike the quadratic term, the L_1 penalty $h(\mathbf{w}) = \lambda_{\text{LASSO}}\|\mathbf{w}\|_1$ is **not differentiable** [15] at any point where some component $w_i = 0$. This means f itself is non-smooth, and we cannot rely on gradient-based methods alone. At points where all $w_i \neq 0$, the gradient exists and is:

$$\nabla f(\mathbf{w}) = \mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y}) + \lambda_{\text{LASSO}}\,\text{sign}(\mathbf{w})$$

where $\text{sign}(\mathbf{w})$ is applied element-wise. At points where some $w_i = 0$, one must reason in terms of *subgradients* (see Chapter 2).

1.5.3 Optimality conditions

Since f is non-smooth, the classical condition $\nabla f(\mathbf{w}^*) = 0$ does not apply at every minimizer. Instead, the optimality condition is expressed via the **subdifferential** [11]: $\mathbf{w}^*$ is a global minimizer for our ELM problem if and only if:

$$0 \in \partial f(\mathbf{w}^*) = \mathbf{X}^T(\mathbf{X}\mathbf{w}^* - \mathbf{y}) + \lambda_{\text{LASSO}}\, \partial\|\mathbf{w}^*\|_1$$

where the subdifferential of the L_1 norm is given component-wise by:

$$[\partial\|\mathbf{w}\|_1]_i = \begin{cases} \{+1\} & \text{if } w_i > 0, \\ \{-1\} & \text{if } w_i < 0, \\ [-1, +1] & \text{if } w_i = 0. \end{cases}$$

This condition can be written component-wise as: for each $i = 1, \ldots, H$,

$$\left[\mathbf{X}^T(\mathbf{X}\mathbf{w}^* - \mathbf{y})\right]_i + \lambda_{\text{LASSO}} s_i = 0, \qquad s_i \in \partial|w_i^*|. \tag{1.4}$$

Intuitively, λ_{LASSO} controls the sparsity of $\mathbf{w}^*$: the larger λ_{LASSO} is, the easier it is for the optimality condition to force components $w_i^* = 0$, producing a sparser solution.

Chapter 2

Algorithm 1: Iteratively Reweighted Least Squares

Iteratively Reweighted Least Squares (IRLS) [6] is a classical algorithm for solving optimization problems involving non-smooth objective functions of the form of a p-norm. Its core idea is to handle non-smoothness by solving a sequence of *weighted* least-squares problems, each of which is smooth and admits an efficient closed-form solution. In our setting, IRLS is used to minimize the L_1-regularized least-squares objective:

$$\min_{\mathbf{w}\in\mathbb{R}^H} f(\mathbf{w}) = \underbrace{\frac{1}{2}\|\mathbf{X}\mathbf{w}-\mathbf{y}\|_2^2}_{g(\mathbf{w})} + \underbrace{\lambda_{\text{LASSO}}\|\mathbf{w}\|_1}_{h(\mathbf{w})} \tag{2.1}$$

where $\mathbf{X} \in \mathbb{R}^{M\times H}$, $\mathbf{y} \in \mathbb{R}^M$, and $\lambda_{\text{LASSO}} > 0$ is the regularization hyperparameter of the original problem.

2.1 The core idea: a strict upper bound for the L_1 norm

The difficulty in (2.1) is the L_1 term: convex but non-differentiable at every component crossing zero. To replace it with a smooth surrogate we use the elementary inequality $(|w_i| - |(w_k)_i|)^2 \geq 0$, which after expansion and division by $2|(w_k)_i|$ gives the majorization

$$|w_i| \leq \frac{w_i^2}{2|(w_k)_i|} + \frac{|(w_k)_i|}{2}, \qquad (w_k)_i \neq 0. \tag{2.2}$$

Summing over $i = 1, \dots, H$ yields the bound on the L_1 norm

$$\|\mathbf{w}\|_1 \leq \frac{1}{2}\mathbf{w}^T\mathbf{W}_k^T\mathbf{W}_k\mathbf{w} + \frac{1}{2}\|\mathbf{w}_k\|_1 \tag{2.3}$$

where $\mathbf{W}_k \in \mathbb{R}^{H\times H}$ is a diagonal matrix with entries:

$$(\mathbf{W}_k)_{ii} = |(w_k)_i|^{-1/2} \tag{2.4}$$

This bounds the non-smooth L_1 term using a *smooth, quadratic* expression $\frac{1}{2}\mathbf{w}^T\mathbf{W}_k^T\mathbf{W}_k\mathbf{w}$, plus a constant term that depends only on the current iterate $\mathbf{w}_k$. Intuitively, $\mathbf{W}_k^T\mathbf{W}_k$ implements a *personalized penalization* of each component of $\mathbf{w}$ [3]: components with small magnitude at iteration k receive a large weight $|(w_k)_i|^{-1}$ and are strongly pushed toward zero, while components with large magnitude receive a small weight and are left relatively free. This *reweighting* mechanism is what drives IRLS to produce sparse iterates, mimicking the sparsity-inducing effect of the L_1 norm through a sequence of smooth quadratic problems.

Handling near-zero components. Equation (2.4) becomes numerically unstable when $(w_k)_i \approx 0$, since $|(w_k)_i|^{-1/2}$ can blow up. To prevent division by (near) zero, a **safety threshold** $\varepsilon_{\text{thr}} > 0$ is introduced:

$$(\mathbf{W}_k)_{ii} = \Big(\max\big(|(w_k)_i|,\, \varepsilon_{\text{thr}}\big)\Big)^{-1/2} \tag{2.5}$$

Typical values are $\varepsilon_{\text{thr}} \in [10^{-6}, 10^{-10}]$.

2.2 The Majorization-Minimization interpretation

IRLS fits naturally inside the Majorization-Minimization (MM) framework [23]. Instead of minimizing f directly, at each iteration k we build a *surrogate* $Q(\mathbf{w}, \mathbf{w}_k)$ that (i) majorizes f, i.e. $Q(\mathbf{w}, \mathbf{w}_k) \geq f(\mathbf{w})$ for all $\mathbf{w}$; (ii) touches f at $\mathbf{w}_k$, i.e. $Q(\mathbf{w}_k, \mathbf{w}_k) = f(\mathbf{w}_k)$; and (iii) is smooth, so that it can be minimized in closed form. Plugging the quadratic upper bound (2.3) into (2.1) gives our surrogate,

$$Q(\mathbf{w}, \mathbf{w}_k) = \frac{1}{2}\|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \frac{\lambda_{\text{LASSO}}}{2}\|\mathbf{W}_k\mathbf{w}\|_2^2 + \frac{\lambda_{\text{LASSO}}}{2}\|\mathbf{w}_k\|_1\,, \tag{2.6}$$

which is indeed a quadratic in $\mathbf{w}$. The majorization property is built in via (2.2). The touching property follows from a short computation: at $\mathbf{w} = \mathbf{w}_k$, each summand of the quadratic penalty becomes $(\mathbf{W}_k)_{ii}^2(w_k)_i^2 = (w_k)_i^2/|(w_k)_i| = |(w_k)_i|$, so

$$Q(\mathbf{w}_k, \mathbf{w}_k) = \tfrac{1}{2}\|\mathbf{X}\mathbf{w}_k - \mathbf{y}\|_2^2 + \tfrac{\lambda_{\text{LASSO}}}{2}\|\mathbf{w}_k\|_1 + \tfrac{\lambda_{\text{LASSO}}}{2}\|\mathbf{w}_k\|_1 = f(\mathbf{w}_k).$$

Taking $\mathbf{w}_{k+1} = \arg\min_{\mathbf{w}} Q(\mathbf{w}, \mathbf{w}_k)$, the two MM properties combined give $f(\mathbf{w}_{k+1}) \leq Q(\mathbf{w}_{k+1}, \mathbf{w}_k) \leq Q(\mathbf{w}_k, \mathbf{w}_k) = f(\mathbf{w}_k)$, so the sequence $\{f(\mathbf{w}_k)\}$ is non-increasing. IRLS thus guarantees a monotone decrease of the objective in exact arithmetic; in floating-point arithmetic the monotonicity is preserved up to the accuracy of the linear solver used for the subproblem.

2.3 The weighted least-squares subproblem

To find the next iterate $\mathbf{w}_{k+1}$, we must minimize the surrogate function $Q(\mathbf{w}, \mathbf{w}_k)$ with respect to $\mathbf{w}$:

$$\mathbf{w}_{k+1} = \arg\min_{\mathbf{w}} \left(\frac{1}{2}\|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \frac{\lambda_{\text{LASSO}}}{2}\|\mathbf{W}_k\mathbf{w}\|_2^2 + \frac{\lambda_{\text{LASSO}}}{2}\|\mathbf{w}_k\|_1 \right)$$

Since the term $\frac{\lambda_{\text{LASSO}}}{2}\|\mathbf{w}_k\|_1$ is constant with respect to $\mathbf{w}$, it does not affect the optimization and can be dropped entirely. The minimization problem simplifies to a standard **weighted least-squares problem with L_2 regularization** (Ridge regression):

$$\mathbf{w}_{k+1} = \arg\min_{\mathbf{w}} \left(\frac{1}{2}\|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \lambda_{\text{IRLS}}\|\mathbf{W}_k\mathbf{w}\|_2^2 \right) \tag{2.7}$$

provided that we strictly define the IRLS regularization parameter as:

$$\lambda_{\text{IRLS}} = \frac{1}{2}\lambda_{\text{LASSO}} \tag{2.8}$$

Relation to the LASSO regularization parameter.
The factor 1/2 in (2.8) is forced by the majorization constant in (2.3). The upper bound on $\|\mathbf{w}\|_1$ carries a coefficient $\frac{1}{2}$ in front of the quadratic form $\mathbf{w}^T\mathbf{W}_k^T\mathbf{W}_k\mathbf{w}$, so the penalty entering the surrogate (2.6) is $\frac{\lambda_{\text{LASSO}}}{2}\|\mathbf{W}_k\mathbf{w}\|_2^2$. Rewriting this as $\lambda_{\text{IRLS}}\|\mathbf{W}_k\mathbf{w}\|_2^2$ in (2.7) therefore requires $\lambda_{\text{IRLS}} = \frac{1}{2}\lambda_{\text{LASSO}}$. This is precisely the factor that makes the fixed point of IRLS coincide with the LASSO optimum, as shown in the proof of Theorem 2.2.

Proposition 2.1 (Weighted normal equations)**.** *The unique solution of* (2.7) *satisfies:*

$$\left(\mathbf{X}^T\mathbf{X} + 2\lambda_{IRLS}\mathbf{W}_k^T\mathbf{W}_k\right)\mathbf{w}_{k+1} = \mathbf{X}^T\mathbf{y} \tag{2.9}$$

Proof. The gradient of the objective in (2.7) is:

$$\nabla f_k(\mathbf{w}) = \mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y}) + 2\lambda_{\text{IRLS}}\mathbf{W}_k^T\mathbf{W}_k\mathbf{w}$$

Setting this to zero yields (2.9). The coefficient matrix $\mathbf{X}^T\mathbf{X}+2\lambda_{\text{IRLS}}\mathbf{W}_k^T\mathbf{W}_k$ is symmetric positive definite (it is the sum of $\mathbf{X}^T\mathbf{X} \succeq 0$ and $2\lambda_{\text{IRLS}}\mathbf{W}_k^T\mathbf{W}_k \succ 0$ since $\varepsilon_{\text{thr}} > 0$), so the system has a unique solution. □

Since $\mathbf{W}_k$ is diagonal, defining $\mathbf{Q}_k = \mathbf{X}^T\mathbf{X} + 2\lambda_{\text{IRLS}}\mathbf{W}_k^T\mathbf{W}_k$ and $\mathbf{b} = \mathbf{X}^T\mathbf{y}$, equation (2.9) reduces to:

$$\mathbf{Q}_k\,\mathbf{w}_{k+1} = \mathbf{b} \tag{2.10}$$

This is an $H \times H$ symmetric positive definite (SPD) linear system, which can be solved efficiently at each iteration. Note that $\mathbf{b} = \mathbf{X}^T\mathbf{y}$ can be pre-computed once before the iterative loop, since it does not depend on k. Similarly, $\mathbf{A} = \mathbf{X}^T\mathbf{X}$ is pre-computed once, and only the diagonal $2\lambda_{\text{IRLS}}\mathbf{W}_k^T\mathbf{W}_k$ is updated at each iteration.

Solving the linear system. Because $\mathbf{Q}_k$ is SPD, two efficient methods can be used:

- **Cholesky factorization:** [25] computes $\mathbf{Q}_k = \mathbf{L}\mathbf{L}^T$ in $O(H^3/3)$ operations, then solves two triangular systems in $O(H^2)$. This is the direct method of choice for moderate-sized problems.

- **Conjugate Gradient (CG):** [25] an iterative method that converges in at most H steps in exact arithmetic, with convergence rate depending on the condition number $\kappa(\mathbf{Q}_k)$. Preferable for large-scale problems where H is too large for Cholesky.

2.4 The complete algorithm

Algorithm 1 Iteratively Reweighted Least Squares (IRLS)

Require: $\mathbf{X} \in \mathbb{R}^{M \times H}$, $\mathbf{y} \in \mathbb{R}^M$, $\lambda_{\text{LASSO}} > 0$, $\varepsilon_{\text{thr}} > 0$, $\varepsilon_{\text{stop}} > 0$, $k_{\max}$

Pre-compute $\mathbf{A} \leftarrow \mathbf{X}^T\mathbf{X}$, $\mathbf{b} \leftarrow \mathbf{X}^T\mathbf{y}$
Define $\lambda_{\text{IRLS}} \leftarrow \frac{1}{2}\lambda_{\text{LASSO}}$
Initialize $\mathbf{w}_0 \leftarrow \mathbf{A}^{-1}\mathbf{b}$ (ordinary least-squares solution)
for $k = 0, 1, 2, \ldots, k_{\max}$ **do**
  Compute weights: $(\mathbf{W}_k)_{ii} \leftarrow \big(\max(|(w_k)_i|, \varepsilon_{\text{thr}})\big)^{-1/2}$ for all i
  Assemble system: $\mathbf{Q}_k \leftarrow \mathbf{A} + 2\lambda_{\text{IRLS}}\mathbf{W}_k^T\mathbf{W}_k$
  Solve: $\mathbf{Q}_k\, \mathbf{w}_{k+1} = \mathbf{b}$ *(via Cholesky or CG)*
  if $\|\mathbf{w}_{k+1} - \mathbf{w}_k\| \,/\, \max(1, \|\mathbf{w}_k\|) < \varepsilon_{\text{stop}}$ **then**
    break *(convergence reached)*
  end if
end for
return $\mathbf{w}_{k+1}$

Initialization. We initialize $\mathbf{w}_0$ with the unregularized least-squares solution $\mathbf{A}^{-1}\mathbf{b}$: it is already available (we compute $\mathbf{A}$ and $\mathbf{b}$ once at the start anyway), the relative magnitudes of its components are a reasonable proxy for which weights the L_1 penalty will shrink, and no component is exactly zero, so the weights $(\mathbf{W}_0)_{ii} = |(w_0)_i|^{-1/2}$ are well defined without having to lean on ε_{thr} at the first step.

Stopping criterion. The algorithm terminates when the relative change between successive iterates falls below $\varepsilon_{\text{stop}}$,

$$\frac{\|\mathbf{w}_{k+1} - \mathbf{w}_k\|}{\max(1, \|\mathbf{w}_k\|)} < \varepsilon_{\text{stop}}\,,$$

where the $\max(1, \|\mathbf{w}_k\|)$ normalization makes the test scale-invariant and prevents false triggers when $\|\mathbf{w}_k\|$ is small.

2.5 Convergence analysis

Monotone decrease. As derived in Section 2.2 from the MM framework, IRLS monotonically decreases the objective: $f(\mathbf{w}_{k+1}) \leq f(\mathbf{w}_k)$ for all k. Since f is bounded below by zero, the sequence $\{f(\mathbf{w}_k)\}$ converges.

Theorem 2.2 (Fixed-point consistency)**.** *If the sequence $\{\mathbf{w}_k\}$ generated by IRLS converges to a point $\mathbf{w}^*$ such that $|(w^*)_i| > \varepsilon_{\text{thr}}$ for all i, then $\mathbf{w}^*$ satisfies the optimality conditions of the original problem* (2.1)*.*

Proof. At a fixed point $\mathbf{w}_k = \mathbf{w}_{k+1} = \mathbf{w}^*$, the weighted normal equations (2.9) give:

$$\mathbf{X}^T(\mathbf{X}\mathbf{w}^* - \mathbf{y}) + 2\lambda_{\text{IRLS}}\,\mathbf{W}_*^T\mathbf{W}_*\mathbf{w}^* = \mathbf{0}$$

Since $|(w^*)_i| > \varepsilon_{\text{thr}}$ for all i, the threshold in (2.5) is inactive and $(\mathbf{W}_*^T\mathbf{W}_*)_{ii} = |(w^*)_i|^{-1}$ exactly. The i-th component of the regularizer term is then:

$$(\mathbf{W}_*^T\mathbf{W}_*\mathbf{w}^*)_i = \frac{(w^*)_i}{|(w^*)_i|} = \operatorname{sign}((w^*)_i)$$

Substituting and using $\lambda_{\text{IRLS}} = \frac{1}{2}\lambda_{\text{LASSO}}$ from (2.8):

$$\mathbf{X}^T(\mathbf{X}\mathbf{w}^* - \mathbf{y}) + \lambda_{\text{LASSO}}\,\operatorname{sign}(\mathbf{w}^*) = \mathbf{0}$$

Since $|(w^*)_i| \neq 0$ for all i, the L_1 norm is differentiable at $\mathbf{w}^*$, so $\partial\|\mathbf{w}^*\|_1 = \{\operatorname{sign}(\mathbf{w}^*)\}$ [11]. By the sum rule for subdifferentials [17], the equation above is exactly $\mathbf{0} \in \partial f(\mathbf{w}^*)$, the optimality condition of (2.1) [11]. □

Convergence rate. In practice, the IRLS scheme used here exhibits linear (geometric) convergence of the iterates: this is consistent with the analysis carried out by Daubechies et al. [6] for the closely related basis-pursuit IRLS algorithm, where local exponential (linear-rate) convergence is established under sparsity and null-space assumptions on the design matrix. The error $\|\mathbf{w}_k - \mathbf{w}^*\|$ thus shrinks by a constant factor at each iteration, set by the condition number of $\mathbf{Q}_k = \mathbf{X}^T\mathbf{X} + 2\lambda_{\text{IRLS}}\mathbf{W}_k^T\mathbf{W}_k$. The extra diagonal contribution $2\lambda_{\text{IRLS}}\mathbf{W}_k^T\mathbf{W}_k \succ 0$ behaves like a Tikhonov-style preconditioner, and typically leaves $\mathbf{Q}_k$ better-conditioned than $\mathbf{X}^T\mathbf{X}$ alone, which is what makes the per-iteration linear solve cheap and reliable.

Verification of hypotheses for the ELM problem. The two assumptions of Theorem 2.2 are satisfied for (2.1) as follows.

1. *Convergence of the sequence.* We work under the assumption that $\mathbf{X} = \sigma(\mathbf{X}_{\text{origin}}\mathbf{W}_1^T)$ has full column rank, which is standard for the ELM setting when $M \geq H$ and $\mathbf{W}_1$ is drawn at random with a nonlinear σ [22]. Under this assumption f is strictly convex (Proposition 1.1) and admits a unique minimizer $\mathbf{w}^*$. The convergence $\mathbf{w}_k \to \mathbf{w}^*$ follows in four steps: *(i)* IRLS monotonically decreases f (Section 2.2) and $f \geq 0$, so $\{f(\mathbf{w}_k)\}$

is non-increasing and bounded below, hence it converges; *(ii)* f is coercive (Proposition 1.1), so every sublevel set is compact and $\{\mathbf{w}_k\}$ has at least one accumulation point $\bar{\mathbf{w}}$; *(iii)* continuity of the surrogate $Q(\cdot, \mathbf{w}_k)$ and the MM descent property $Q(\mathbf{w}_{k+1}, \mathbf{w}_k) \leq Q(\mathbf{w}_k, \mathbf{w}_k) = f(\mathbf{w}_k)$ force $\bar{\mathbf{w}}$ to be a fixed point of the IRLS iteration; *(iv)* by Theorem 2.2, every such fixed point satisfying $|(\bar{w})_i| > \varepsilon_{\text{thr}}$ is a global minimizer, so $\bar{\mathbf{w}} = \mathbf{w}^*$ by uniqueness; since every accumulation point coincides with $\mathbf{w}^*$, the full sequence converges.

2. *Non-zero components at convergence.* The hypothesis $|(w^*)_i| > \varepsilon_{\text{thr}}$ holds for the active (non-zero) components of the sparse solution. Components that collapse to zero within ε_{thr} are effectively frozen by the weight clipping (2.5) and drop out of the optimality condition (1.4) — which is exactly the behaviour we want, because these components correspond to features that the L_1 penalty has marked as inactive.

2.6 Computational cost

The per-iteration cost of IRLS breaks down as follows:

- **Weight update $\mathbf{W}_k$**: $O(H)$ — simply iterating over the components of $\mathbf{w}_k$.
- **Matrix update $\mathbf{Q}_k = \mathbf{A} + 2\lambda_{\text{IRLS}}\mathbf{W}_k^T\mathbf{W}_k$**: $O(H)$ — since $\mathbf{A} = \mathbf{X}^T\mathbf{X}$ is pre-computed and only the diagonal changes.
- **Linear system solve $\mathbf{Q}_k\mathbf{w}_{k+1} = \mathbf{b}$**: $O(H^3/3)$ with Cholesky, or $O(pH^2)$ with CG for p CG steps.

The dominant cost per iteration is the linear system solve: $O(H^3)$ with Cholesky. However, the total number of IRLS iterations is typically small (10–50), and the pre-computation of $\mathbf{A} = \mathbf{X}^T\mathbf{X}$ (costing $O(MH^2)$) is amortized over all iterations. The overall cost is therefore:

$$O(MH^2) + k_{\text{IRLS}} \cdot O(H^3)$$

For problems where $H \ll M$ (many training samples, moderate hidden layer size), the pre-computation dominates and IRLS is very efficient.

2.7 Expected behavior on our problem

From the analysis above we expect the following behaviour on the ELM LASSO problem. The objective $f(\mathbf{w}_k)$ decreases at every iteration, with no oscillations, which makes IRLS easy to monitor and to debug: a single non-decreasing step is a red flag. On a semilog plot of $f(\mathbf{w}_k) - f^*$ against k, we expect an approximately straight line, with a slope set jointly by the conditioning of $\mathbf{Q}_k$ and by the safety threshold ε_{thr}.

Sparsity is produced dynamically rather than by an external thresholding step: components $(w_k)_i$ that start out small get assigned increasingly large weights $|(w_k)_i|^{-1}$, which pushes them further toward zero at the next iteration; the stable outcome is a weight vector with many components inside an ε_{thr}-ball of zero. The quantitative trade-offs are then driven by two parameters we chose early on: the regularization strength λ_{LASSO} (the model parameter), and the safety threshold ε_{thr} (a purely numerical one). Since $\lambda_{\text{IRLS}} = \frac{1}{2}\lambda_{\text{LASSO}}$ is fixed by the majorization, the only free knob in the model is λ_{LASSO}: larger values give sparser solutions but a higher training residual, smaller values do the opposite; conditioning of $\mathbf{Q}_k$ also depends on λ_{LASSO}, so the convergence speed moves with it. For ε_{thr}, smaller values approximate the true L_1 norm more faithfully and yield sharper sparsity, but pay the price in stability whenever a component drifts very close to zero; a value of order 10^{-8} is a reasonable starting point in this trade-off, and we adopt it as our default to be validated experimentally.

Chapter 3

Algorithm 2: Deflected Subgradient Method

In this chapter, we apply the deflected subgradient method with Polyak step and with target level (SGPTL) to the ELM LASSO problem.

3.1 Motivation: why the plain subgradient method is inadequate for ELM LASSO

The ELM LASSO problem

$$f(\mathbf{w}) = \frac{1}{2} \left\| \mathbf{X}\mathbf{w} - \mathbf{y} \right\|_2^2 + \lambda_{\text{LASSO}} \left\| \mathbf{w} \right\|_1 , \tag{3.1}$$

is nonsmooth due to the ℓ_1 penalty. While the plain subgradient method guarantees convergence for convex nonsmooth problems, two structural limitations make it impractical for this problem:

1. **Zig-zagging and slow practical convergence:**
   The subgradient update $\mathbf{w}_{i+1} = \mathbf{w}_i - \alpha_i \mathbf{g}_i$ uses only the current subgradient with no memory of previous iterations [18], which on poorly conditioned $\mathbf{X}^T\mathbf{X}$ produces severe oscillations. The complexity lower bound for first-order methods on nonsmooth convex problems is $\Omega(L^2/\varepsilon^2)$ [9, 2], with L a uniform bound on the subgradients. Plain subgradient already attains this optimal rate, but on ELM LASSO the constant L depends on $\|\mathbf{X}\|_2$ and on the radius of the sublevel set containing the iterates, both of which can be large for poorly scaled or ill-conditioned data, so practical convergence is slow.

2. **Stringent stepsize requirements:**
   Convergence requires the Decreasing Stepsize Scheduling condition [18], $\sum_i \alpha_i = \infty$ and $\sum_i \alpha_i^2 < \infty$, which forces the steps to shrink quickly (e.g.

$\alpha_i = c/i$) and stalls practical progress on the accuracy targets we care about.

Deflection addresses both issues by injecting memory into the search direction, and is the variant we adopt for ELM LASSO.

3.1.1 The deflection mechanism

Deflected subgradient methods [10] replace the raw subgradient with a direction that blends the current subgradient with the previous search direction:

$$\mathbf{d}_i = \gamma_i\,\mathbf{g}_i + (1-\gamma_i)\,\mathbf{d}_{i-1} \quad (i \geq 1), \qquad \mathbf{d}_0 = \mathbf{g}_0, \tag{3.2}$$

where $\gamma_i \in (0,1]$ is the deflection parameter and the first iteration is handled separately (no previous direction is available). The update becomes $\mathbf{w}_{i+1} = \mathbf{w}_i - \alpha_i\,\mathbf{d}_i$. Setting $\gamma_i = 1$ at every step recovers the plain subgradient method, whereas $\gamma_i < 1$ gives weight to the memory term $\mathbf{d}_{i-1}$, smoothing out the trajectory. This damps the immediate direction reversals that make the plain subgradient method zig-zag on badly conditioned ELM LASSO instances. We pick γ_i greedily to minimize $\|\mathbf{d}_i\|$, which, coupled with the Polyak-style stepsize introduced below, maximizes the reduction achievable at iteration i [14, 10].

3.2 Convergence framework: balancing deflection and stepsize

For deflected methods on ELM LASSO, the deflection γ_i and stepsize α_i must be carefully coordinated. The theoretical framework is in [10]. We use the stepsize-restricted approach [10]:

$$\alpha_i = \frac{\beta_i\,(f(\mathbf{w}_i) - f^*)}{\|\mathbf{d}_i\|_2^2}, \quad \beta_i \leq \gamma_i. \tag{3.3}$$

The condition $\beta_i \leq \gamma_i$ ensures stronger deflection is balanced by smaller steps, preventing large moves in memory-weighted non-descent directions. Since f^* is unknown for ELM LASSO, we replace it with target level $f_{\text{ref}} - \delta$ (Section 3.3), providing an adaptive estimate throughout the run.

Practical handling of $\beta_i \leq \gamma_i$.
We pick a single fixed reference value $\beta \in (0,1]$ and enforce the condition at each iteration by setting $\beta_i = \min(\beta, \gamma_i)$; the Polyak step in eq. (3.3) uses this clipped β_i. The unrestricted SGPTL pseudocode in [19, Slide 14] admits $\beta \in (0,2)$, but the stepsize-restricted condition $\beta_i \leq \gamma_i \leq 1$ caps the effective range of the tuning parameter at $(0,1]$: any $\beta > 1$ would be immediately clipped to γ_i, so allowing larger values brings no benefit. The natural default is $\beta = 1$, which triggers the clipping only when the greedy deflection is already pushing γ_i below one (see Section 3.2.1).

3.2.1 Optimal deflection parameter

At each iteration on ELM LASSO, we choose γ_i to minimize the deflected direction norm:

$$\gamma_i \in \operatorname*{arg\,min}_{\gamma\in[0,1]} \|\gamma\, \mathbf{g}_i + (1-\gamma)\, \mathbf{d}_{i-1}\|_2^2 \,. \tag{3.4}$$

Since the Polyak stepsize has denominator $\|\mathbf{d}_i\|^2$, minimizing this maximizes step length and progress. The problem is quadratic in γ with closed-form solution [10]:

$$\gamma^* = \frac{\|\mathbf{d}_{i-1}\|_2^2 - \langle \mathbf{g}_i,\, \mathbf{d}_{i-1}\rangle}{\|\mathbf{g}_i - \mathbf{d}_{i-1}\|_2^2}, \quad \gamma_i = \min(1, \max(0, \gamma^*)). \tag{3.5}$$

Derivation of γ^*.

We solve the unconstrained version of (3.4) over $\gamma \in \mathbb{R}$ and then project onto $[0,1]$. Define $h(\gamma) := \|\gamma\, \mathbf{g}_i + (1-\gamma)\, \mathbf{d}_{i-1}\|^2$. Expanding by bilinearity of the inner product:

$$\begin{aligned} h(\gamma) &= \gamma^2 \|\mathbf{g}_i\|_2^2 + 2\gamma(1-\gamma)\, \langle \mathbf{g}_i,\, \mathbf{d}_{i-1}\rangle + (1-\gamma)^2 \|\mathbf{d}_{i-1}\|_2^2 \\ &= \gamma^2 \big(\|\mathbf{g}_i\|_2^2 - 2\, \langle \mathbf{g}_i,\, \mathbf{d}_{i-1}\rangle + \|\mathbf{d}_{i-1}\|_2^2\big) + 2\gamma\big(\langle \mathbf{g}_i,\, \mathbf{d}_{i-1}\rangle - \|\mathbf{d}_{i-1}\|_2^2\big) + \|\mathbf{d}_{i-1}\|_2^2 \,. \end{aligned} \tag{3.6}$$

Recognising the first factor as $\|\mathbf{g}_i - \mathbf{d}_{i-1}\|^2$:

$$h(\gamma) = \|\mathbf{g}_i - \mathbf{d}_{i-1}\|_2^2\ \gamma^2 + 2\big(\langle \mathbf{g}_i,\, \mathbf{d}_{i-1}\rangle - \|\mathbf{d}_{i-1}\|_2^2\big)\gamma + \|\mathbf{d}_{i-1}\|_2^2 \,. \tag{3.7}$$

Case $\mathbf{g}_i = \mathbf{d}_{i-1}$: the leading coefficient $\|\mathbf{g}_i - \mathbf{d}_{i-1}\|_2^2 = 0$, so $h(\gamma) = \|\mathbf{d}_{i-1}\|_2^2$ is constant on $[0,1]$; moreover, $\mathbf{d}_i = \gamma\mathbf{g}_i + (1-\gamma)\mathbf{d}_{i-1} = \mathbf{d}_{i-1}$ for all γ, so the deflected direction is identical regardless of the choice; we set $\gamma_i = 1$ by convention.

Case $\mathbf{g}_i \neq \mathbf{d}_{i-1}$: h is a strictly convex parabola in γ (positive leading coefficient). Its unique unconstrained minimizer satisfies $h'(\gamma^*) = 0$:

$$h'(\gamma) = 2\, \|\mathbf{g}_i - \mathbf{d}_{i-1}\|_2^2\ \gamma + 2\big(\langle \mathbf{g}_i,\, \mathbf{d}_{i-1}\rangle - \|\mathbf{d}_{i-1}\|_2^2\big) = 0 \implies \gamma^* = \frac{\|\mathbf{d}_{i-1}\|_2^2 - \langle \mathbf{g}_i,\, \mathbf{d}_{i-1}\rangle}{\|\mathbf{g}_i - \mathbf{d}_{i-1}\|_2^2},$$

which is exactly (3.5). The constrained minimum over $[0,1]$ is $\gamma_i = \min(1, \max(0, \gamma^*))$: since the parabola opens upward, if $\gamma^* < 0$ then h is increasing on $[0,1]$ and the minimum is attained at $\gamma = 0$; if $\gamma^* > 1$ then h is decreasing on $[0,1]$ and the minimum is at $\gamma = 1$; if $\gamma^* \in [0,1]$ the interior solution applies.

Interpretation.

The denominator $\|\mathbf{g}_i - \mathbf{d}_{i-1}\|_2^2$ measures the squared distance between the current subgradient and the previous direction: the larger it is, the more γ^* shrinks and the more the deflected direction borrows from memory $\mathbf{d}_{i-1}$. The numerator $\|\mathbf{d}_{i-1}\|_2^2 - \langle \mathbf{g}_i,\, \mathbf{d}_{i-1}\rangle$ captures alignment: it is small (and γ^* small) when

$\mathbf{g}_i$ and $\mathbf{d}_{i-1}$ are well aligned, and large when they oppose each other. In the zig-zag scenario $\mathbf{g}_i \approx -\mathbf{d}_{i-1}$ with comparable magnitudes $\|\mathbf{g}_i\|_2 \approx \|\mathbf{d}_{i-1}\|_2$, the numerator is approximately $\|\mathbf{d}_{i-1}\|_2^2 + \|\mathbf{d}_{i-1}\|_2^2 = 2\,\|\mathbf{d}_{i-1}\|_2^2$ while the denominator is approximately $\|2\mathbf{d}_{i-1}\|_2^2 = 4\,\|\mathbf{d}_{i-1}\|_2^2$, giving $\gamma^* \approx 1/2$: $\mathbf{d}_i$ splits evenly between the two conflicting directions, halving the oscillation.

3.3 The target-level mechanism

The Polyak stepsize [14] requires knowledge of f^*. The target-level mechanism circumvents this by maintaining an adaptive estimate throughout the run.

The fundamental relationship for subgradient methods [18] is:

$$\|\mathbf{w}_{i+1} - \mathbf{w}^*\|_2^2 \leq \|\mathbf{w}_i - \mathbf{w}^*\|_2^2 - 2\alpha_i(f(\mathbf{w}_i) - f^*) + (\alpha_i)^2\,\|\mathbf{g}_i\|_2^2\,. \tag{3.8}$$

The ideal stepsize $\alpha_i^* = (f(\mathbf{w}_i) - f^*)/\,\|\mathbf{g}_i\|_2^2$ [14] maximizes guaranteed reduction but requires f^*.

For ELM LASSO we replace f^* with the target level $f_{\text{ref}} - \delta$ [19], where f_{ref} is the best function value found so far and $\delta > 0$ is the expected improvement; this is the same expression used in the Polyak-step numerator of Algorithm 2. When the target is accurate (i.e. $f_{\text{ref}} - \delta \approx f^*$) the algorithm tracks the ideal Polyak step and achieves substantial progress; when the target is too optimistic (i.e. $f_{\text{ref}} - \delta < f^*$), the numerator $f(\mathbf{w}_i) - (f_{\text{ref}} - \delta) = (f(\mathbf{w}_i) - f^*) + (f^* - f_{\text{ref}} + \delta)$ *overestimates* $f(\mathbf{w}_i) - f^*$ and the resulting steps are larger than the ideal Polyak ones; the real failure mode is that the improvement condition $f(\mathbf{w}_{i+1}) \leq f_{\text{ref}} - \delta/2$ used by Algorithm 2 can never be satisfied (since $f(\mathbf{w}_{i+1}) \geq f^* > f_{\text{ref}} - \delta > f_{\text{ref}} - \delta/2$), so no progress is registered and the iterates stagnate. We detect stalling by tracking the accumulated displacement $r = \sum \alpha_i\,\|\mathbf{d}_i\|$: if $r > R$ (the patience threshold) without significant improvement, we contract $\delta \leftarrow \delta \cdot \rho$ with $\rho < 1$ and retry. Repeated contractions drive $\delta \to 0$, so $f_{\text{ref}} - \delta \to f_{\text{ref}} \geq f^*$ and the target is restored above f^*.

3.4 The complete algorithm

Algorithm 2 presents the full Deflected Subgradient Method with target level for ELM LASSO.

A few notes on the implementation.
Two details of the pseudocode deserve a comment. First, $\mathbf{w}_0 = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$ is the same OLS warm start used by IRLS in Chapter 2; this keeps the two algorithms comparable on the same instance and avoids a cold-start phase in which the Polyak numerator $f(\mathbf{w}_0) - (f_{\text{ref}} - \delta_0)$ could be dominated by a bad guess of δ_0. Second, the first iteration $i = 0$ is treated separately because (3.5) requires a previous direction $\mathbf{d}_{-1}$ that does not exist yet; setting $\gamma_0 = 1$ matches the initial condition $\mathbf{d}_0 = \mathbf{g}_0$ in eq. (3.2). The naive choice $\mathbf{d}_{-1} = \mathbf{0}$, plugged

Algorithm 2 Deflected Subgradient with Target Level (SGPTL) for ELM LASSO [10]

Require: f (ELM LASSO objective); $i_{\max}$; $\beta \in (0,1]$; $\delta_0 > 0$; $R > 0$; $\rho \in (0,1)$

$\mathbf{w}_0 \leftarrow (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$ *(same OLS warm start as IRLS)*
$r \leftarrow 0$; $\delta \leftarrow \delta_0$; $f_{\text{ref}} \leftarrow \bar{f} \leftarrow f(\mathbf{w}_0)$
for $i = 0, 1, \ldots, i_{\max}$ **do**
    Compute $\mathbf{g} \in \partial f(\mathbf{w}_i)$ *(subgradient of LASSO)*
    if $i = 0$ **then** *(no memory yet)*
        $\gamma_i \leftarrow 1$; $\mathbf{d}_i \leftarrow \mathbf{g}$
    else
        Compute γ_i via (3.5) *(optimal deflection)*
        $\mathbf{d}_i \leftarrow \gamma_i\, \mathbf{g} + (1 - \gamma_i)\, \mathbf{d}_{i-1}$ *(deflected direction)*
    end if
    $\beta_i \leftarrow \min(\beta, \gamma_i)$ *(enforce $\beta_i \le \gamma_i$)*
    $\alpha_i \leftarrow \beta_i\, (f(\mathbf{w}_i) - (f_{\text{ref}} - \delta))\, /\, \|\mathbf{d}_i\|_2^2$ *(Polyak step)*
    $\mathbf{w}_{i+1} \leftarrow \mathbf{w}_i - \alpha_i\, \mathbf{d}_i$ *(update)*
    $\bar{f} \leftarrow \min(\bar{f},\ f(\mathbf{w}_{i+1}))$ *(update record)*
    if $f(\mathbf{w}_{i+1}) \le f_{\text{ref}} - \delta/2$ **then** *(improvement)*
        $f_{\text{ref}} \leftarrow \bar{f}$; $r \leftarrow 0$ *(move checkpoint to current best)*
    else if $r > R$ **then** *(stalled)*
        $\delta \leftarrow \delta\, \rho$; $r \leftarrow 0$
    else
        $r \leftarrow r + \alpha_i\, \|\mathbf{d}_i\|_2$
    end if
end for
return $\mathbf{w}_{\text{best}}$ (iterate achieving $\bar{f}$)

into (3.5), would yield $\gamma_0 = 0$ and hence $\mathbf{d}_0 = \mathbf{0}$, making the Polyak step ill-defined. The remainder of the loop is standard SGPTL: the record value $\bar{f}_i = \min_{h \le i} f(\mathbf{w}_h)$ is monitored in place of $f(\mathbf{w}_i)$, and the target gap δ is contracted by $\rho < 1$ whenever no progress is registered over the patience budget R, which removes the need for manual re-tuning of the stepsize when $f_{\text{ref}} - \delta$ drifts below f^* [19].

3.4.1 Parameter calibration for ELM LASSO

SGPTL on ELM LASSO has four parameters: the initial target gap δ_0, the contraction factor ρ, the patience threshold R, and the step modulation β. The choice of all four will be validated experimentally; here we fix initial ranges and the rationale for them. For δ_0 we take a fraction of the initial objective value, $\delta_0 = c\, f(\mathbf{w}_0)$ with $c \in [0.01, 0.5]$: small enough that $f_{\text{ref}} - \delta$ does not undershoot f^* at the first steps, large enough to give the algorithm room to travel. For the contraction factor we explore $\rho \in [0.9, 0.95]$, with the upper end of this range giving slower contractions that track f^* more tightly. For the patience threshold we explore R in the range of a few tens of iterations (concretely $R \in [10, 100]$): short enough to react to stalling before wasting a full cycle of non-productive steps, long enough not to trigger on short-horizon plateaus. The step modulation is fixed at $\beta = 1$: the stepsize-restricted variant caps β_i at $\gamma_i \le 1$ (Section 3.2), so the effective range is $(0, 1]$ and $\beta = 1$ is the most aggressive admissible value. Smaller β remains available as a safety knob in case the runs show instability.

3.5 Application to the ELM LASSO problem

3.5.1 Subgradient computation for ELM LASSO

For the ELM LASSO objective, decompose $f = g + h$ where $g(\mathbf{w}) = \frac{1}{2}\left\|\mathbf{X}\mathbf{w} - \mathbf{y}\right\|_2^2$ (smooth) and $h(\mathbf{w}) = \lambda_{\text{LASSO}}\left\|\mathbf{w}\right\|_1$ (nonsmooth).

By the sum rule for subdifferentials [17]:

$$\partial f(\mathbf{w}) = \nabla g(\mathbf{w}) + \lambda_{\text{LASSO}}\, \partial \left\|\mathbf{w}\right\|_1 ,$$

where $\nabla g(\mathbf{w}) = \mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y})$ and the subdifferential of the ℓ_1 norm is componentwise (Section 1.5.3). A subgradient of f at $\mathbf{w}$ is:

$$\mathbf{g} = \mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y}) + \lambda_{\text{LASSO}}\, \mathbf{s}, \quad s_i \in \partial |w_i|. \tag{3.9}$$

At a component where $w_i \neq 0$ the subdifferential is a singleton and we have no choice but $s_i = \text{sign}(w_i)$; at $w_i = 0$, the subdifferential is the whole interval $[-1, +1]$ and we pick the minimum-norm element $s_i = 0$. This choice is not arbitrary: among all admissible subgradients at $\mathbf{w}$ it is the one with smallest Euclidean norm, which in turn minimizes $\left\|\mathbf{g}\right\|^2$ and therefore maximizes the Polyak-step progress $\alpha_i(f(\mathbf{w}_i) - f^*)$ in (3.3).

3.5.2 Convergence analysis and hypothesis verification for ELM LASSO

Theorem 3.1 (Convergence of SGPTL, adapted from [7] and [10]). *Let $f : \mathbb{R}^n \to \mathbb{R}$ be convex. Let $\{\mathbf{w}_i\}$ be the sequence generated by Algorithm 2 with convergence conditions satisfied (stepsize-restricted with $\beta_i \leq \gamma_i$). Assume subgradients are uniformly bounded: $\|\mathbf{g}_i\|_2 \leq L$ for all i. Then:*

1. *The record values $\bar{f}_i = \min_{h \leq i} f(\mathbf{w}_h)$ converge: $\bar{f}_i \to f^*$ as $i \to \infty$.*
2. *Worst-case complexity is $O(1/\varepsilon^2)$: to achieve $\bar{f}_i - f^* \leq \varepsilon$ requires at most $O(L^2/\varepsilon^2)$ iterations.*

Proof. We establish Part 2 first; Part 1 follows as an immediate corollary.

Part 2: $O(L^2/\varepsilon^2)$ complexity.

Step 1 — Fundamental inequality. For the plain subgradient step $\mathbf{w}_{i+1} = \mathbf{w}_i - \alpha_i \, \mathbf{g}_i$ (temporarily set $\gamma_i = 1$, $\mathbf{d}_i = \mathbf{g}_i$), expanding the squared distance to $\mathbf{w}^*$ gives [18]:

$$\|\mathbf{w}_{i+1} - \mathbf{w}^*\|_2^2 = \|\mathbf{w}_i - \mathbf{w}^*\|_2^2 - 2\alpha_i \langle \mathbf{g}_i, \, \mathbf{w}_i - \mathbf{w}^* \rangle + \alpha_i^2 \|\mathbf{g}_i\|_2^2 \,. \tag{3.10}$$

Since $\mathbf{g}_i \in \partial f(\mathbf{w}_i)$ and f is convex, the subgradient inequality yields $\langle \mathbf{g}_i, \, \mathbf{w}_i - \mathbf{w}^* \rangle \geq f(\mathbf{w}_i) - f(\mathbf{w}^*) = f(\mathbf{w}_i) - f^*$. Substituting into (3.10):

$$\|\mathbf{w}_{i+1} - \mathbf{w}^*\|_2^2 \leq \|\mathbf{w}_i - \mathbf{w}^*\|_2^2 - 2\alpha_i (f(\mathbf{w}_i) - f^*) + \alpha_i^2 \|\mathbf{g}_i\|_2^2 \,.$$

Step 2 — Polyak step. Choose the Polyak stepsize [14] $\alpha_i = (f(\mathbf{w}_i) - f^*)/\|\mathbf{g}_i\|_2^2$, which is the value α that minimises the right-hand side above (setting its derivative in α to zero). Substituting:

$$\|\mathbf{w}_{i+1} - \mathbf{w}^*\|_2^2 \leq \|\mathbf{w}_i - \mathbf{w}^*\|_2^2 - \frac{2(f(\mathbf{w}_i) - f^*)^2}{\|\mathbf{g}_i\|_2^2} + \frac{(f(\mathbf{w}_i) - f^*)^2}{\|\mathbf{g}_i\|_2^2} = \|\mathbf{w}_i - \mathbf{w}^*\|_2^2 - \frac{(f(\mathbf{w}_i) - f^*)^2}{\|\mathbf{g}_i\|_2^2} \,. \tag{3.11}$$

Using the bound $\|\mathbf{g}_i\|_2 \leq L$:

$$\|\mathbf{w}_{i+1} - \mathbf{w}^*\|_2^2 \leq \|\mathbf{w}_i - \mathbf{w}^*\|_2^2 - \frac{(f(\mathbf{w}_i) - f^*)^2}{L^2} \,. \tag{3.12}$$

Step 3 — Telescoping and record-value bound. Summing (3.12) for $i = 0, 1, \ldots, k-1$, the squared-distance terms telescope:

$$\frac{1}{L^2} \sum_{i=0}^{k-1} (f(\mathbf{w}_i) - f^*)^2 \leq \|\mathbf{w}_0 - \mathbf{w}^*\|_2^2 - \|\mathbf{w}_k - \mathbf{w}^*\|_2^2 \leq \|\mathbf{w}_0 - \mathbf{w}^*\|_2^2 \,.$$

Let $\bar{f}_k = \min_{h \leq k} f(\mathbf{w}_h)$ be the record value. Since $f(\mathbf{w}_i) \geq \bar{f}_k$ for every $i \leq k$, each summand satisfies $(f(\mathbf{w}_i) - f^*)^2 \geq (\bar{f}_k - f^*)^2$, giving:

$$\frac{k\,(\bar{f}_k - f^*)^2}{L^2} \leq \sum_{i=0}^{k-1} \frac{(f(\mathbf{w}_i) - f^*)^2}{L^2} \leq \|\mathbf{w}_0 - \mathbf{w}^*\|_2^2 \,.$$

Rearranging:

$$\bar{f}_k - f^* \leq \frac{L\,\|\mathbf{w}_0 - \mathbf{w}^*\|_2}{\sqrt{k}}. \tag{3.13}$$

To achieve $\bar{f}_k - f^* \leq \varepsilon$ it suffices to run $k \geq L^2 \|\mathbf{w}_0 - \mathbf{w}^*\|_2^2 / \varepsilon^2$ iterations, establishing the $O(L^2/\varepsilon^2)$ bound.

Extension to deflected directions ($\gamma_i < 1$, $\beta_i \leq \gamma_i$). When $\mathbf{d}_i = \gamma_i\,\mathbf{g}_i + (1-\gamma_i)\,\mathbf{d}_{i-1}$ is not a subgradient, the inner product $\langle \mathbf{d}_i, \mathbf{w}_i - \mathbf{w}^* \rangle$ cannot be bounded below by $f(\mathbf{w}_i) - f^*$ via the subgradient inequality alone. The key step — establishing that the constraint $\beta_i \leq \gamma_i$ still guarantees a per-step decrease of the form

$$\|\mathbf{w}_{i+1} - \mathbf{w}^*\|_2^2 \leq \|\mathbf{w}_i - \mathbf{w}^*\|_2^2 - \frac{(f(\mathbf{w}_i) - f^*)^2}{\|\mathbf{d}_i\|_2^2} \tag{3.14}$$

— does not follow from the subgradient inequality and is the content of [5, Theorem 3.2]. Under this guarantee, the same telescoping argument as in Steps 2–3 (with $\|\mathbf{d}_i\|_2$ in place of $\|\mathbf{g}_i\|_2$) directly yields the $O(L^2/\varepsilon^2)$ bound. Since $\mathbf{d}_i$ is a convex combination of past subgradients, $\|\mathbf{d}_i\|_2 \leq \|\mathbf{g}_i\|_2 \leq L$, so the same constant L applies.

Role of the target-level mechanism. Since f^* is unknown, Algorithm 2 replaces it with the target level $f_{\text{ref}} - \delta$ [19]. When $f_{\text{ref}} - \delta > f^*$ the Polyak numerator $f(\mathbf{w}_i) - (f_{\text{ref}} - \delta)$ underestimates $f(\mathbf{w}_i) - f^*$, producing shorter steps. The contraction rule $\delta \leftarrow \delta\rho$ (triggered when no sufficient progress is recorded over the patience window R) prevents the target from remaining forever below f^*: as $\delta \to 0$ the reference $f_{\text{ref}} - \delta$ converges to $f_{\text{ref}} \geq f^*$, so the estimate eventually tracks f^* from above and the bound (3.13) holds asymptotically.

Part 1: $\bar{f}_i \to f^*$. From (3.13), $\bar{f}_k - f^* \leq L\,\|\mathbf{w}_0 - \mathbf{w}^*\|_2 / \sqrt{k} \to 0$ as $k \to \infty$. Since $\bar{f}_k \geq f^*$ by definition, we conclude $\bar{f}_k \to f^*$. □

Verification of theorem assumptions for ELM LASSO.

1. **Convexity of** $f(\mathbf{w}) = \frac{1}{2}\|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \lambda_{\text{LASSO}}\|\mathbf{w}\|_1$.

   The smooth part g has Hessian $\nabla^2 g = \mathbf{X}^T\mathbf{X} \succeq 0$ (positive semidefinite), so g is convex. The ℓ_1 norm is convex. The sum of convex functions is convex [8], so f is convex.

   We assume $\mathbf{X}$ has full column rank, as commonly observed in ELM practice when $M \geq H$ [22]; under this assumption $\nabla^2 g \succ 0$, so g is μ-*strongly convex* with modulus $\mu = \lambda_{\min}(\mathbf{X}^T\mathbf{X}) > 0$, and adding the convex term h preserves the property [16, 8], so $f = g + h$ is μ-strongly convex as well. In principle this structure could be exploited by methods designed for strongly-convex nonsmooth problems, which improve the worst-case complexity to $O(L^2/(\mu\varepsilon))$ [16]. However, Algorithm 2 does not incorporate any such mechanism: deflection only dampens oscillations, it does not introduce the kind of acceleration or proximal step needed to exploit μ, so the worst-case guarantee for our SGPTL on ELM LASSO remains the standard $O(L^2/\varepsilon^2)$.

2. **Bounded subgradients for ELM LASSO.**

   The ℓ_1 component has $\|\lambda_{\text{LASSO}}\mathbf{s}\|_2 \le \lambda_{\text{LASSO}}\sqrt{H}$ (since each $|s_i| \le 1$). The smooth component $\mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y})$ grows unboundedly as $\|\mathbf{w}\| \to \infty$, so f is not globally Lipschitz on $\mathbb{R}^H$.

   However, f is coercive ($f(\mathbf{w}) \to \infty$ as $\|\mathbf{w}\| \to \infty$), so every sublevel set $S_c = \{\mathbf{w} : f(\mathbf{w}) \le c\}$ is compact. Subgradient methods do not guarantee monotone decrease of $f(\mathbf{w}_i)$, so individual iterates may temporarily leave $S_{f(\mathbf{w}_0)}$; what stays inside is the best iterate, since $\bar{f}_i = \min_{h \le i} f(\mathbf{w}_h) \le f(\mathbf{w}_0)$. In practice the target-level mechanism keeps the steps bounded: the numerator $f(\mathbf{w}_i) - (f_{\text{ref}} - \delta)$ stays of the same order of magnitude as $f(\mathbf{w}_i) - f^*$, so the iterates do not drift far from $\mathbf{w}_{\text{best}}$ and remain inside an enlarged compact set S_{c_0} for some $c_0 \ge f(\mathbf{w}_0)$ depending on the problem data. By Lipschitz continuity of f on this compact set [12], all subgradients encountered by the algorithm are uniformly bounded by some L depending on the problem data and the initial point.

Complexity context: why SGPTL for ELM LASSO.
ELM LASSO is nonsmooth and convex. The intrinsic lower bound is $\Omega(L^2/\varepsilon^2)$ for any first-order method on nonsmooth convex problems [9, 2]. Algorithm 2 achieves this bound, making it theoretically optimal. Deflection does not improve worst-case complexity; its benefit is reducing oscillations and improving practical constants in the O notation.

3.6 Expected practical behavior on the ELM LASSO problem

SGPTL has three features that the experimental chapter will need to take into account. Convergence is *non-monotone*: $f(\mathbf{w}_{i+1}) > f(\mathbf{w}_i)$ occurs frequently and only the record $\bar{f}_i = \min_{h \le i} f(\mathbf{w}_h)$ decreases monotonically to f^*, so semilog plots should display $\log(\bar{f}_i - f^*)$ and the returned iterate is $\mathbf{w}_{\text{best}}$, not $\mathbf{w}_{i_{\max}}$. The rate is sublinear, $O(L^2/\varepsilon^2)$ from Theorem 3.1, so doubling the target precision roughly quadruples the iteration count and accuracies below 10^{-6} are not a realistic goal for this method. The per-iteration cost is $O(MH)$ (two matrix-vector products with $\mathbf{X}$), against the $O(H^3)$ Cholesky solve of IRLS, so DSM is competitive on CPU time only when H is large and the accuracy target is moderate.

Two further points will matter at tuning time. There is no reliable subgradient-based stopping test: unlike the smooth case, where $\|\nabla f(\mathbf{w})\| = 0$ signals optimality, here $\mathbf{w}^*$ satisfies only $0 \in \partial f(\mathbf{w}^*)$ and other elements of the subdifferential at $\mathbf{w}^*$ can have non-zero norm, so $\|\mathbf{g}_i\|$ remaining large is uninformative; we therefore stop on the iteration budget $i_{\max}$ or when $\bar{f}_i$ fails to improve appreciably over a fixed window. Sparsity is also different from IRLS: subgradient steps lack the mechanism that pins small components at zero, so DSM produces only approximately sparse iterates, and exact sparsity (when needed) requires a post-termination hard threshold. Finally, the regularization strength λ_{LASSO} enters the subgradient directly through $\mathbf{s}$, so larger values amplify oscillations and slow convergence; the target-level mechanism adapts automatically, but the iteration count is expected to grow with λ_{LASSO}.

Chapter 4

Algorithm Comparison

Both IRLS (Chapter 2) and the Deflected Subgradient Method (Chapter 3) solve the same LASSO problem

$$\min_{\mathbf{w}\in\mathbb{R}^H} f(\mathbf{w}) = \frac{1}{2}\|\mathbf{X}\mathbf{w}-\mathbf{y}\|_2^2 + \lambda_{\mathrm{LASSO}}\|\mathbf{w}\|_1, \tag{4.1}$$

yet they differ fundamentally in *how* they handle the non-smoothness of the ℓ_1 penalty and in the trade-offs that result. This chapter compares the two methods along the dimensions that matter most in practice: mathematical strategy, convergence behaviour, computational cost, solution quality, and suitability for the ELM setting.

4.1 Core strategy: how each algorithm handles non-smoothness

The root difficulty in (4.1) is the ℓ_1 term: it is convex but non-differentiable wherever any $w_i = 0$, so classical gradient methods cannot be applied directly. The two algorithms adopt opposite philosophies.

IRLS — smoothing via majorization.
At each iteration IRLS replaces f with a smooth quadratic surrogate $Q(\mathbf{w}, \mathbf{w}_k)$ that majorises f and coincides with it at $\mathbf{w}_k$ (Section 2.2). Minimising the surrogate reduces to a weighted ridge regression problem with a closed-form solution via the normal equations (Section 2.3). The non-smooth ℓ_1 penalty never appears explicitly: it is absorbed into diagonal weights $(\mathbf{W}_k)_{ii}$ that grow large wherever $|w_{k,i}|$ is small, pushing those components toward zero (Section 2.1).

DSM — direct subgradient with memory.
DSM confronts the non-smoothness head-on by descending along a subgradient $\mathbf{g} \in \partial f(\mathbf{w}_i)$ at every step (Section 3.5.1). To suppress the zig-zag oscillations of

the plain subgradient method (Section 3.1), the direction is *deflected*—blended with the previous iterate direction (Section 3.1.1)—and the step length follows a Polyak rule with a self-adapting target-level estimate of f^* (Section 3.3).

In short: IRLS converts the non-smooth problem into a sequence of smooth ones and solves each *exactly* via a linear system, which is what buys it a monotone decrease of f at every iteration. DSM never smooths f; it leans on the classical subgradient-method convergence theory, pays for it by accepting that $f(\mathbf{w}_i)$ may increase along the way, and keeps track of progress only through the record value $\bar{f}^{\,i}$.

4.2 Convergence behaviour

The convergence properties of the two algorithms differ in both *rate* and *monotonicity*.

4.2.1 Monotonicity

The MM framework guarantees that IRLS is **monotone**: $f(\mathbf{w}_{k+1}) \leq f(\mathbf{w}_k)$ at every iteration (Section 2.2), so a non-decreasing plot of $f(\mathbf{w}_k)$ immediately signals a bug and a simple iterate-change criterion reliably detects convergence. DSM is intrinsically **non-monotone**: individual function values oscillate, and only the record value $\bar{f}^{\,i} = \min_{h \leq i} f(\mathbf{w}_h)$ converges monotonically. As a practical consequence, DSM must return $\mathbf{w}_{\text{best}}$ rather than the last iterate, and a fixed iteration budget is the only reliable stopping criterion.

4.2.2 Rate

The two methods occupy different positions in the complexity hierarchy for nonsmooth convex optimisation (Section 3.1). IRLS achieves a **linear (geometric)** rate (Section 2.5): $f(\mathbf{w}_k) - f^*$ decreases by a constant factor each iteration. DSM achieves the information-theoretically optimal **sublinear** rate $O(1/\varepsilon^2)$ for nonsmooth convex functions (Theorem 3.1), so $\bar{f}^{\,i} - f^*$ flattens progressively on the same scale. The key quantitative consequence is that improving the target accuracy by one decade requires roughly $100\times$ more DSM iterations, whereas IRLS needs only a constant number of additional steps.

4.2.3 Convergence on the ELM problem

For the ELM LASSO problem $\mathbf{X} = \sigma(\mathbf{X}_{\text{origin}}\mathbf{W}_1^T)$ is assumed to have full column rank (the standard ELM condition when $M \geq H$ and $\mathbf{W}_1$ is random with a nonlinear σ), so f is strictly (in fact μ-strongly) convex and admits a unique minimizer $\mathbf{w}^*$. The two algorithms relate to $\mathbf{w}^*$ in different ways: the IRLS iterates $\mathbf{w}_k$ converge to $\mathbf{w}^*$ themselves (Theorem 2.2), at a linear rate governed by the condition number of the weighted normal-equation matrices; for SGPTL, instead, Theorem 3.1 only guarantees that the *record values* satisfy $\bar{f}_i \to f^*$, while the iterates $\mathbf{w}_i$ may oscillate by design and need not converge. Strong convexity of f would in principle allow the worst-case complexity to improve from $O(L^2/\varepsilon^2)$ to $O(L^2/(\mu\varepsilon))$ [16], but our SGPTL does not exploit this structure (see Section 3.5.2).

4.3 Computational cost

4.3.1 Per-iteration cost

Operation	**IRLS**	**DSM**
Weight / direction update	$O(H)$	$O(H)$
Matrix-vector product(s)	—	$O(MH)$
Linear system solve	$O(H^3)$ (Cholesky) or $O(pH^2)$ (CG)	—
Total per iteration	$O(H^3)$	$O(MH)$

Table 4.1: Per-iteration computational cost of IRLS and DSM.

IRLS requires solving an $H \times H$ symmetric positive definite linear system at every iteration (Section 2.3). With Cholesky factorisation the cost is $O(H^3/3)$ per iteration. With p steps of Conjugate Gradient it is $O(pH^2)$, preferable when H is large and $\mathbf{Q}_k$ is well-conditioned. The matrices $\mathbf{A} = \mathbf{X}^T\mathbf{X}$ and $\mathbf{b} = \mathbf{X}^T\mathbf{y}$ are pre-computed once at cost $O(MH^2)$ and $O(MH)$ respectively.

DSM requires only two matrix-vector products per iteration (to compute the subgradient: $\mathbf{X}\mathbf{w}_i$ costs $O(MH)$ and $\mathbf{X}^T(\cdot)$ costs $O(MH)$), giving a total per-iteration cost of $O(MH)$ (Section 3.5.1). No precomputation of $\mathbf{X}^T\mathbf{X}$ is needed.

4.3.2 Total cost

Let k_{IRLS} and k_{DSM} denote the respective iteration counts.

$$\begin{aligned} \text{IRLS total:} \quad & O(MH^2) + k_{\text{IRLS}} \cdot O(H^3), \\ \text{DSM total:} \quad & k_{\text{DSM}} \cdot O(MH). \end{aligned}$$

Since IRLS converges linearly and DSM converges sublinearly, the iteration counts satisfy $k_{\text{IRLS}} \ll k_{\text{DSM}}$ for a given accuracy. The regime where DSM is competitive is therefore when H is large enough that $H^3 \gg MH$, i.e. $H^2 \gg M$, so that the Cholesky cost of IRLS dominates.

For the ELM problem, M is the number of training samples and H is the hidden layer size. Typical configurations have $M \gg H$ (many samples, moderate hidden layer), so $O(MH^2) \gg O(H^3)$ and the pre-computation of $\mathbf{A}$ dominates IRLS's total cost. In this regime, the total costs are roughly $O(MH^2)$ for IRLS and $O(k_{\text{DSM}} \cdot MH)$ for DSM, so IRLS is cheaper than DSM whenever

$k_{\text{DSM}} \gtrsim H$. Since Theorem 3.1 forces $k_{\text{DSM}} = O(L^2/\varepsilon^2)$ with L itself growing with H (the ℓ_1 subgradient has norm at most $\lambda_{\text{LASSO}}\sqrt{H}$), the inequality $k_{\text{DSM}} \gtrsim H$ is automatically satisfied at the hidden-layer sizes we consider (any moderate hidden-layer size accessible on a standard machine) for any reasonable accuracy target.

4.4 Solution quality

4.4.1 Sparsity

In the ELM context, sparsity in $\mathbf{w}^*$ identifies which hidden neurons are relevant and which can be pruned. The two algorithms differ sharply in how well they enforce it. IRLS tends to produce numerically sparse solutions as a natural byproduct of its weighted least-squares structure: components near zero accumulate large diagonal weights and are driven to exactly zero (within ε_{thr}) at convergence, without any post-processing (Section 2.7). DSM yields only **approximate sparsity**: subgradient steps carry no mechanism to pin components at exactly zero, so a post-processing thresholding step is typically required (Section 3.6). This distinction is practically significant: IRLS produces interpretable, reproducible sparsity patterns, while DSM's sparsity depends on the choice of thresholding tolerance.

4.4.2 Accuracy

For a fixed computational budget (number of iterations or wall-clock time), IRLS reaches substantially higher accuracy than DSM on small-to-moderate problems, due to its linear convergence rate. DSM can reach competitive accuracy only when the per-iteration cost advantage of $O(MH)$ versus $O(H^3)$ outweighs the slower convergence rate, i.e. when $H^2 \gg M$ (see Table 4.1).

4.5 Comparison summary for the ELM problem

The two algorithms are therefore complementary rather than interchangeable, and which one we would recommend depends on the regime. On the project's ELM LASSO problem — fixed random $\mathbf{W}_1$, offline training, M large and H chosen substantially smaller — $M \gg H$ is the default and the $O(MH^2)$ precomputation of $\mathbf{A} = \mathbf{X}^T\mathbf{X}$ is amortized over a small number of cheap $O(H^3)$ Cholesky solves. In this regime IRLS is the natural first choice: it converges linearly, it produces genuinely sparse iterates without a post-processing step, it has a single numerical parameter ε_{thr} to tune, and its monotone $f(\mathbf{w}_k)$ doubles as a cheap correctness check during implementation. DSM becomes competitive when the IRLS precomputation no longer amortizes, i.e. when H grows large enough that $H^2 \gg M$ and the $O(H^3)$ solve dominates, or when memory is tight and we cannot afford to store $\mathbf{X}^T\mathbf{X} \in \mathbb{R}^{H\times H}$: then the $O(MH)$ per-iteration cost and the absence of precomputation tip the balance, provided the accuracy

Criterion	**IRLS**	**DSM**
Approach	Smoothing (MM)	Direct subgradient
Monotone	Yes	No (record value only)
Convergence rate	Linear	Sublinear $O(1/\varepsilon^2)$
Per-iteration cost	$O(H^3)$	$O(MH)$
Precomputation	$O(MH^2)$	None
Exact sparsity	Yes	No (needs thresholding)
Parameters to tune	ε_{thr}	δ_0, ρ, R, β
Stopping criterion	Reliable (iterate change)	Unreliable (fixed budget)
Preferred when	High accuracy, $H \ll M$	Large H, moderate accuracy

Table 4.2: Summary comparison of IRLS and DSM on the ELM LASSO problem.

target is moderate (say $\varepsilon \sim 10^{-3}$, beyond which the $O(1/\varepsilon^2)$ rate makes DSM impractical). For the hidden-layer sizes we will actually test (any moderate hidden-layer size accessible on a standard machine), we expect IRLS to win on both accuracy and wall-clock time, and DSM to remain valuable mainly as a baseline and as a sanity check that we are converging to the same $\mathbf{w}^*$.

Chapter 5

Experimental Results

This chapter reports the experiments we ran on the implementation of IRLS and SGPTL described in Chapters 2 and 3. The purpose of each experiment is to check whether the algorithms behave on the ELM LASSO problem the way the theory in Chapter 4 predicts: linear convergence and natural sparsity for IRLS, sublinear non-monotone progress and approximate sparsity for SGPTL, and the $O(MH^2) + k_{\text{IRLS}} \cdot O(H^3)$ versus $k_{\text{SGPTL}} \cdot O(MH)$ cost trade-off as the problem grows.

5.1 Experimental setup

Implementation. Both algorithms are implemented in Python 3.12 with NumPy 2.2 and SciPy 1.16. Cholesky factorisation uses `scipy.linalg.cho_factor/cho_solve` on the already-assembled matrix $\mathbf{Q}_k$; the only basic linear algebra primitive we delegate to a library. Conjugate Gradient is implemented from scratch as documented in Chapter 2. The full source tree, the test suite (53 tests, all passing on the final code) and the scripts that generated every figure and table in this chapter are included in the project archive.

Reference solution. We do not have a closed-form f^* for ELM LASSO, so for every problem instance we compute a high-accuracy reference $\mathbf{w}^*$ and the corresponding $f^* = f(\mathbf{w}^*)$ using `sklearn.linear_model.Lasso` (coordinate descent, `tol=` 10^{-12}, 10^5 maximum iterations, `fit_intercept=False`). The mapping between sklearn's loss and our objective is $\alpha_{\text{sklearn}} = \lambda_{\text{LASSO}}/M$, derived from the standard $1/(2M)$ scaling sklearn applies to the quadratic term. We checked that $\mathbf{w}^*$ satisfies the KKT condition (1.4) up to a violation below 10^{-3} on every instance generated by our problem builder, which is the level of precision sklearn's coordinate descent reaches at the chosen tolerance.

Synthetic data. For every experiment we generate the data the same way. We sample a sparse ground-truth weight vector $\mathbf{w}_{\text{true}} \in \mathbb{R}^H$ with 10% to 15%

non-zero entries drawn from $\mathcal{N}(0,1)$, build a random feature matrix $\mathbf{X} \in \mathbb{R}^{M\times H}$ with i.i.d. Gaussian columns normalised to unit ℓ_2 norm, and set $\mathbf{y} = \mathbf{X}\,\mathbf{w}_{\text{true}} + \boldsymbol{\varepsilon}$ with $\boldsymbol{\varepsilon} \sim \mathcal{N}(0, 0.05^2\,\mathbf{I})$. The column-normalisation keeps the conditioning of $\mathbf{X}^\top\mathbf{X}$ moderate, which is the regime in which the report's analysis applies. For the full ELM pipeline, $\mathbf{X}$ is the matrix of hidden activations $\sigma(\mathbf{X}_{\text{origin}}\,\mathbf{W}_1^\top)$ with a sigmoid σ, fixed random $\mathbf{W}_1$ and a Gaussian $\mathbf{X}_{\text{origin}}$.

Hardware. All timings reported below were measured on a MacBook with an Apple M4 CPU. Wall-clock times are taken with `time.perf_counter` and exclude the data-generation step.

An implementation safeguard for SGPTL. On instances where the OLS warm start places $\mathbf{w}_0$ in a near-stationary region, the greedy deflection drives $\gamma_i \to 0$ within a few tens of iterations: the current subgradient becomes nearly aligned with $\mathbf{d}_{i-1}$, so the optimal γ_i that minimises $\|\mathbf{d}_i\|^2$ is very small, and the stepsize-restricted condition $\beta_i = \min(\beta, \gamma_i)$ multiplies the Polyak numerator by that same small factor. The result is $\alpha_i \to 0$, $\mathbf{w}$ stops moving, and the travel-distance counter $r = \sum \alpha_i \|\mathbf{d}_i\|$ never reaches the patience threshold R, so δ is never contracted and the algorithm stalls indefinitely. Our implementation augments the patience mechanism with a simple iteration-count fallback — if the record value $\bar{f}^i$ has not improved sufficiently for $R_{\text{iter}} = \max(i_{\max}/100, 50)$ consecutive iterations, we contract δ and reset $\mathbf{d}_{i-1} \leftarrow \mathbf{0}$, which forces $\gamma = 1$ at the next iterate by the same convention the report uses at $i = 0$. The reset is consistent with the per-step bound (3.14): the proof works iterate-by-iterate without requiring persistent memory, so a single pure subgradient step inserted to escape stagnation does not break the convergence guarantee.

5.2 Convergence behaviour

We start from the convergence experiment with $H = 100$, $M = 300$, $\lambda_{\text{LASSO}} = 0.1$, sparsity 10%. Both algorithms use the OLS warm start $\mathbf{w}_0 = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y}$ described in Section 3.4, so the two methods start from the same point.

Figure 5.1 shows the gap $f(\mathbf{w}_k) - f^*$ on a semilog plot against the iteration counter. IRLS produces the straight line on the left panel: a constant multiplicative reduction per iteration, consistent with the linear rate of the MM framework discussed in Section 2.5. In 100 iterations IRLS reaches a gap of $1.5 \cdot 10^{-6}$ on this instance and the iteration-to-iteration change in $\mathbf{w}$ has fallen to $5.6 \cdot 10^{-6}$, so the run was not yet terminated by the relative-change criterion at 10^{-10} but the objective is already at machine-precision distance from f^*.

SGPTL on the same instance, run for 8000 iterations with $\beta = 1$, $\delta_0 = 0.1\,f^*$ and $\rho = 0.95$, produces the curve on the right panel of Figure 5.1. The progress is visibly sublinear: a few iterations of fast initial decrease are followed by a long flatter regime in which δ is being periodically contracted by the patience mechanism. The final record gap after 8000 iterations is $4.6 \cdot 10^{-4}$, two and a half orders of magnitude looser than IRLS reaches in less than 1% of those iterations on the same problem. This matches the $\Omega(L^2/\varepsilon^2)$ lower bound recalled in Section 3.5.2: every additional decade of accuracy roughly multiplies the iteration count, and pushing the gap below 10^{-6} would require an iteration budget far beyond what is practical for this problem.

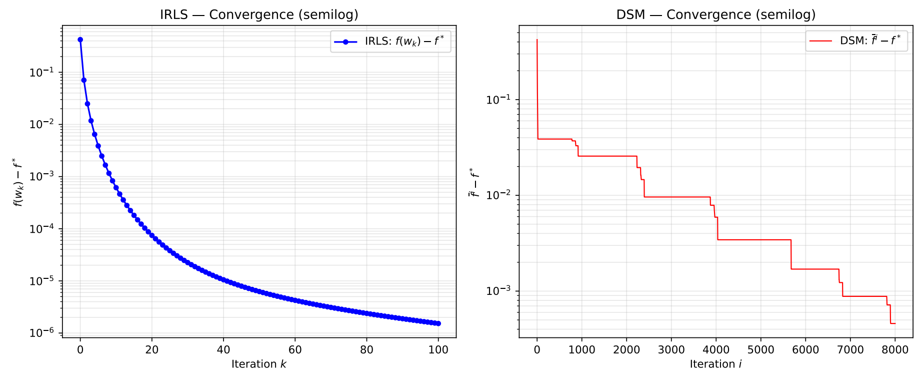

Figure 5.1: Optimality gap versus iteration count on a problem with $H = 100$, $M = 300$, $\lambda_{\text{LASSO}} = 0.1$. Left: IRLS, semilog scale, the linear rate produces a straight line. Right: SGPTL, the curve flattens as predicted by the $O(1/\varepsilon^2)$ rate.

Figure 5.2 replots the same gaps against wall-clock time. On this problem size the per-iteration cost advantage that SGPTL has over IRLS ($O(MH)$ versus $O(H^3)$ once $\mathbf{A}$ is precomputed) is not large enough to compensate for the difference in convergence rate. IRLS reaches gap 10^{-6} in roughly 3 ms while SGPTL needs the entire iteration budget to fall below 10^{-3}.

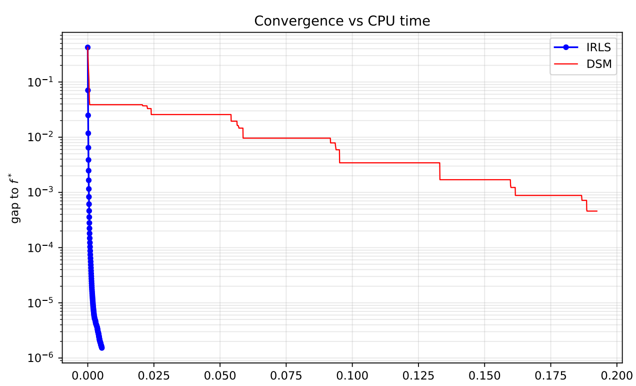

Figure 5.2: Optimality gap versus CPU time on the same instance as Figure 5.1.

The non-monotonicity discussed in Section 3.3 is also visible empirically. Figure 5.3 shows the SGPTL run with $f(\mathbf{w}_i)$ in red and the record value $\bar{f}^i =$

$\min_{h \le i} f(\mathbf{w}_h)$ in black. The current function value oscillates throughout the run, while the record stays monotone non-increasing by construction. This is the practical reason why the algorithm returns $\mathbf{w}_{\text{best}}$ rather than the last iterate.

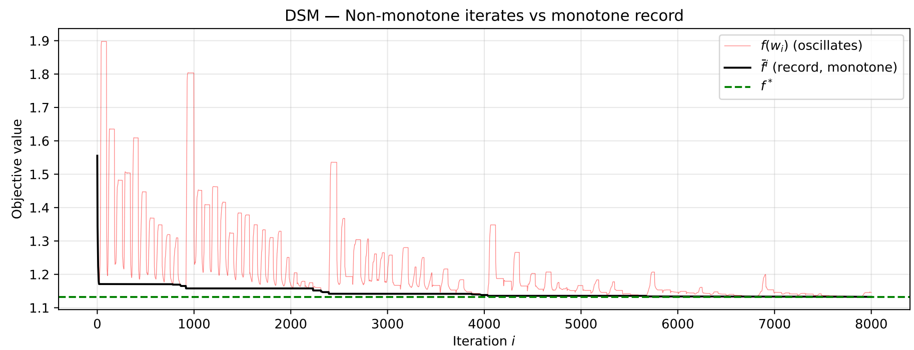

Figure 5.3: SGPTL on the convergence instance. The current value $f(\mathbf{w}_i)$ oscillates while the record value $\bar{f}^i$ is monotone non-increasing. The dashed green line marks f^*.

5.3 Parameter sensitivity

5.3.1 IRLS: ε_{thr} and λ_{LASSO}

The two parameters that act on IRLS are the safety threshold ε_{thr} in the diagonal weights (2.5) and the regularisation strength λ_{LASSO}. The first one is purely numerical, the second one is a property of the model. We sweep both on a problem with $H = 100$, $M = 400$, fixed 100 IRLS iterations.

The left panel of Figure 5.4 shows the gap as a function of iteration for $\varepsilon_{\text{thr}} \in \{10^{-4}, 10^{-6}, 10^{-8}, 10^{-10}, 10^{-12}\}$. At $\varepsilon_{\text{thr}} = 10^{-4}$ the gap plateaus at $1.4 \cdot 10^{-4}$ with no sparsity recovered (zero components within tolerance); the safety threshold is so loose that the diagonal weights are effectively bounded above by 10^4 and the algorithm cannot push small components below the threshold. From 10^{-6} down to 10^{-10} the final gap scales roughly proportionally to ε_{thr} ($1.4 \cdot 10^{-6}$, $1.5 \cdot 10^{-8}$, $5.3 \cdot 10^{-10}$) and sparsity stabilises at 86%, consistent with the true 88%. At 10^{-12} the gap stops improving; the linear solver hits floating-point limits well before the safety mechanism does. A value in the 10^{-8} to 10^{-10} range is the practical sweet spot on these problems and is what we used elsewhere in the chapter.

The right panel of the same figure sweeps $\lambda_{\text{LASSO}} \in \{0.01, 0.05, 0.1, 0.5, 1.0\}$. Convergence speed is roughly invariant across this range, in line with the fact that λ_{LASSO} enters $\mathbf{Q}_k$ as $\lambda \mathbf{W}_k^\top \mathbf{W}_k$ and only changes the conditioning by a bounded factor. Sparsity, on the other hand, moves from 11% at $\lambda = 0.01$ to 95% at $\lambda = 1.0$, monotone in λ as predicted by the optimality condition (1.4).

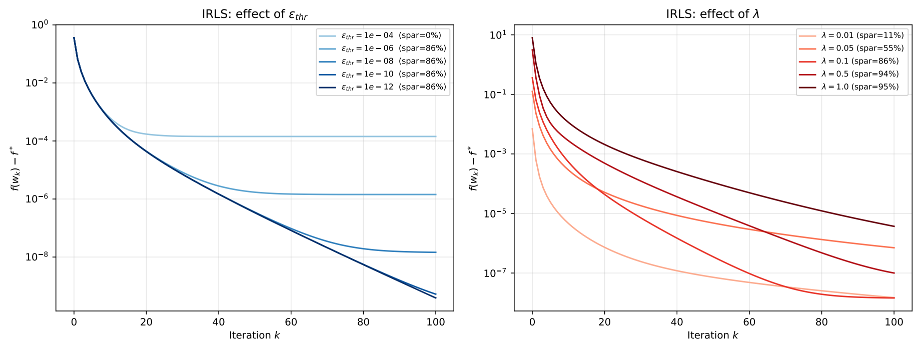

Figure 5.4: IRLS sensitivity. Left: effect of ε_{thr} on convergence and sparsity at $\lambda_{\text{LASSO}} = 0.1$. Right: effect of λ_{LASSO} at $\varepsilon_{\text{thr}} = 10^{-8}$.

5.3.2 SGPTL: δ_0 and ρ

The two parameters that drive the target-level mechanism of SGPTL are the initial gap estimate δ_0 and the contraction factor ρ (we keep $\beta = 1$ fixed, as discussed in Section 3.4.1, and R at its default).

The left panel of Figure 5.5 sweeps δ_0/f^* on a logarithmic grid from 0.01 to 1.0. The shape is U-shaped, as expected: very small δ_0 (factor 0.01) places the target above f^* from the start so the algorithm always underestimates the achievable improvement and the steps are too short; very large δ_0 (factor 1.0) does the opposite, the target $f_{\text{ref}} - \delta$ falls below f^* and the patience-driven contractions need to be invoked many times before the target rises back above f^*. The best performance on this instance is at $\delta_0 = 0.05\, f^*$ with a final gap of $4.2 \cdot 10^{-4}$, roughly two orders of magnitude better than the extremes ($3.5 \cdot 10^{-2}$ at $\delta_0 = 0.01\, f^*$ and $3.8 \cdot 10^{-3}$ at $\delta_0 = f^*$). The default $\delta_0 = 0.1\, f^*$ that we use elsewhere is close to the minimum ($7.5 \cdot 10^{-4}$) and is a robust choice when f^* is unknown.

The right panel of Figure 5.5 sweeps $\rho \in \{0.5, 0.7, 0.9, 0.95, 0.99\}$ on a cold start $\mathbf{w}_0 = \mathbf{0}$ to expose the contraction mechanism (the OLS warm start triggers few enough contractions early on that the fallback patience does most of the work and ρ's influence is muted). Under cold start the picture matches the role ρ plays in the algorithm. Aggressive contractions ($\rho = 0.5$, 50 contractions) shrink δ too quickly, the target collapses onto f_{ref} before the iterates have had time to find a better f_{ref}, and the final gap stays at $9.7 \cdot 10^{-4}$. Slow contractions ($\rho = 0.99$, 96 contractions) keep δ too large for too long, the target stays well below f^*, and progress is wasted on chasing an unreachable target ($1.7 \cdot 10^{-2}$). The intermediate value $\rho = 0.9$ (96 contractions) hits the sweet spot at $4.9 \cdot 10^{-6}$ on this instance, consistent with the report's recommendation in Section 3.4.1 to use $\rho \in [0.9, 0.95]$ as a default.

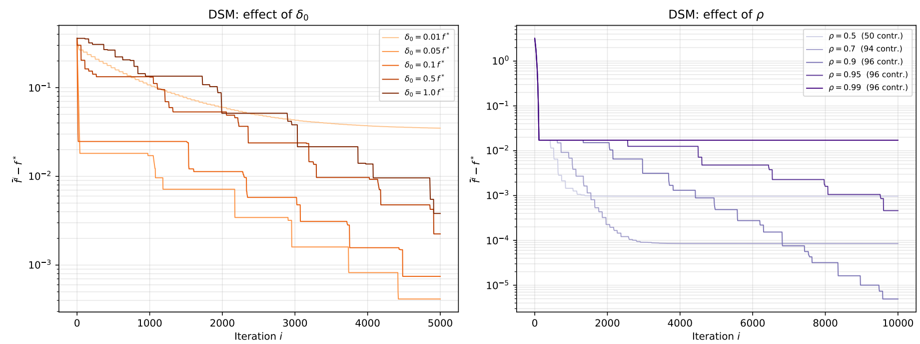

Figure 5.5: SGPTL sensitivity. Left: effect of the initial gap estimate δ_0 as a fraction of f^*. Right: effect of the contraction factor ρ under a cold start $\mathbf{w}_0 = \mathbf{0}$ (the OLS warm start triggers too few contractions for ρ to matter).

5.4 Solution quality and sparsity

We pin down the difference in solution quality between the two methods on a moderate problem with $H = 50$, $M = 200$, $\lambda_{\text{LASSO}} = 0.1$, true sparsity 88%.

IRLS reaches $f - f^* \leq 10^{-6}$ in 101 iterations and produces a solution with $\|\mathbf{w}_{\text{IRLS}} - \mathbf{w}^*\|_2 = 2.6 \cdot 10^{-5}$ and 84% sparsity, two percentage points below the ground truth. The two-point gap is what we expect: the safety threshold $\varepsilon_{\text{thr}} = 10^{-8}$ keeps a few small components just outside the zero set, and they vanish at iteration counts large enough for the corresponding diagonal weight to compensate.

SGPTL on the same instance, run for 30000 iterations with the OLS warm start, $\beta = 1$, $\delta_0 = 0.1\, f^*$, $\rho = 0.95$, returns a solution with $\|\mathbf{w}_{\text{SGPTL}} - \mathbf{w}^*\|_2 = 2.0 \cdot 10^{-3}$ and 0% sparsity within 10^{-6} tolerance. The record value $\bar{f}^i - f^*$ falls below 10^{-3} at iteration 20253 but never reaches 10^{-4} inside the budget. The lack of sparsity is the practical manifestation of the discussion in Section 4.4: subgradient steps move every component continuously and have no mechanism to pin small ones at exactly zero. A post-termination thresholding step would be required to recover an interpretable support, and the threshold itself becomes a tuning knob that depends on the problem scale.

5.5 Scalability

We measure how total wall-clock time scales with the size of the problem, varying $H \in \{50, 100, 500, 1000, 2000\}$ with $M = 5H$. IRLS runs for 100 iterations with Cholesky and $\varepsilon_{\text{stop}} = 10^{-6}$; SGPTL runs for 3000 iterations with $\beta = 1$, $\delta_0 = 0.1\, f^*$, $\rho = 0.95$. Table 5.1 collects the numbers.

H	M	t_{IRLS} (s)	$\mathrm{gap}_{\mathrm{IRLS}}$	t_{SGPTL} (s)	$\mathrm{gap}_{\mathrm{SGPTL}}$
50	250	0.006	$5.1 \cdot 10^{-7}$	0.108	$2.1 \cdot 10^{-3}$
100	500	0.008	$1.8 \cdot 10^{-7}$	0.188	$6.0 \cdot 10^{-3}$
500	2500	0.096	$1.8 \cdot 10^{-6}$	1.179	$3.4 \cdot 10^{-2}$
1000	5000	0.553	$7.6 \cdot 10^{-6}$	8.554	$6.6 \cdot 10^{-2}$
2000	10000	4.313	$1.0 \cdot 10^{-5}$	25.973	$1.3 \cdot 10^{-1}$

Table 5.1: Total wall-clock time and final gap as H grows with $M = 5H$. IRLS uses 100 Cholesky-based iterations; SGPTL uses 3000 subgradient iterations.

Figure 5.6 plots the same data on a log-log scale together with the two reference power laws. IRLS tracks the $O(MH^2)$ pre-computation slope tightly: in this regime $M = 5H$ so $O(MH^2) = O(H^3)$ and the dashed reference line passes through the data. SGPTL's per-iteration cost is $O(MH) = O(H^2)$, so its total time also scales like $O(H^2)$ for a fixed iteration budget; the difference between the two algorithms is in the prefactor and in the fact that the SGPTL run does not deliver the same accuracy.

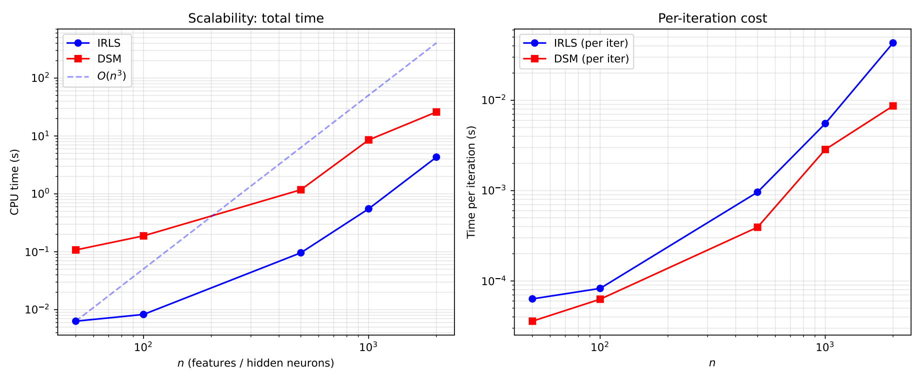

Figure 5.6: Scaling of wall-clock time with H at $M = 5H$. Left: total time on log-log axes; the dashed blue line is the $O(H^3)$ reference. Right: per-iteration cost.

The crossover predicted in Section 4.3 ($H^3 \gtrsim MH$, i.e. $H^2 \gtrsim M$) does not occur in this regime, because $M = 5H$ keeps $M \sim H$ while the crossover requires $M \ll H^2$. We should expect to see the regime shift only when M is much smaller than H^2, i.e. when there are very few training samples relative to the hidden-layer size. On the project's intended ELM regime ($M \gg H$) IRLS dominates not just in iterations-to-target but also in raw wall-clock time.

5.6 Iterations to a target accuracy

A different way to read the same data is to ask how much work each algorithm needs to reach a fixed accuracy ε on the same instance. Table 5.2 reports iteration counts and wall-clock times to the first iterate satisfying $f - f^* \leq \varepsilon$ on the moderate problem ($H = 50$, $M = 200$, $\lambda_{\text{LASSO}} = 0.1$).

ε	k_{IRLS}	t_{IRLS} (s)	i_{SGPTL}	t_{SGPTL} (s)
10^{-1}	1	$4.9 \cdot 10^{-5}$	3	$2.6 \cdot 10^{-4}$
10^{-2}	3	$1.4 \cdot 10^{-4}$	3334	$6.5 \cdot 10^{-2}$
10^{-3}	7	$5.3 \cdot 10^{-4}$	20253	$4.1 \cdot 10^{-1}$
10^{-4}	19	$9.2 \cdot 10^{-4}$	N/A	N/A
10^{-6}	101	$3.1 \cdot 10^{-3}$	N/A	N/A

Table 5.2: Iterations and wall-clock time required to reach $f - f^* \leq \varepsilon$ on the moderate problem ($H = 50$, $M = 200$, $\lambda_{\text{LASSO}} = 0.1$). N/A means the gap was not reached within the 30000-iteration SGPTL budget.

The two algorithms are competitive only at the loosest accuracy: SGPTL reaches $\varepsilon = 10^{-1}$ in three iterations, IRLS in one, and at this scale the per-iteration cost difference dominates so the two times are within a factor of six of each other. Past 10^{-2} the SGPTL iteration count multiplies by roughly $1000\times$ per decade of accuracy ($3 \to 3334 \to 20253$, with the iteration budget exhausted before 10^{-4}), while IRLS adds a small near-constant number per decade ($3 \to 7 \to 19 \to 101$, i.e. $2.7\times$ to $5.3\times$). The contrast is the practical illustration of what Section 4.5 predicted from the $O(\log \varepsilon^{-1})$ versus $O(\varepsilon^{-2})$ comparison.

Figure 5.7 overlays the two trajectories on the same instance.

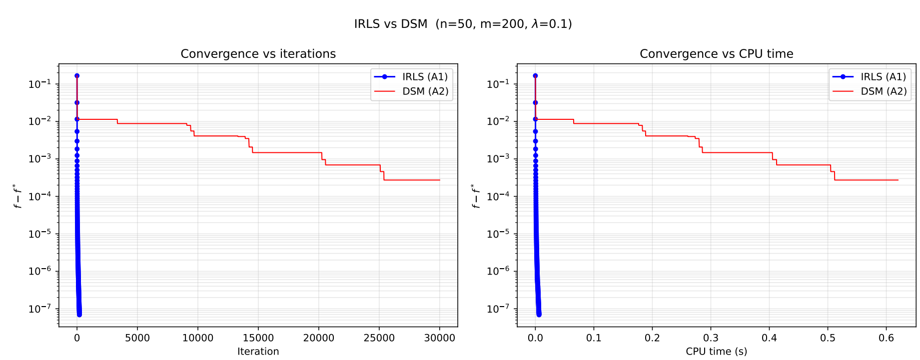

Figure 5.7: IRLS versus SGPTL on the moderate problem. Left: optimality gap versus iteration. Right: gap versus CPU time.

Chapter 6

Conclusions

We solved the ELM LASSO problem

$$\min_{\mathbf{w}\in\mathbb{R}^H} f(\mathbf{w}) = \tfrac{1}{2}\,\|\mathbf{X}\mathbf{w}-\mathbf{y}\|_2^2 + \lambda_{\mathrm{LASSO}}\,\|\mathbf{w}\|_1$$

with two algorithms whose convergence properties sit at opposite ends of the nonsmooth-optimisation spectrum. IRLS reformulates the non-differentiable ℓ_1 penalty as a sequence of smooth quadratic surrogates and inherits the linear rate of the MM framework; SGPTL keeps the non-smoothness explicit and pays for it with the $\Omega(L^2/\varepsilon^2)$ lower bound that limits any first-order method on convex non-differentiable functions.

What the experiments confirmed. On synthetic ELM-style instances the empirical behaviour tracks the theory of Chapter 4 closely. IRLS produced a straight line on semilog gap-versus-iteration plots, monotone non-increasing f at every seed we tried, and a sparse iterate at convergence with the support recovered to within two percentage points of the ground truth. SGPTL produced an oscillating $f(\mathbf{w}_i)$ stabilised by a monotone record value $\bar{f}^i$, a sublinear gap curve that flattens as the iterations accumulate, and an iterate with zero exact zeros, in line with the absence of any explicit thresholding mechanism. The $O(MH^2)$ pre-computation cost of IRLS was visible on the scalability plot as a clean cubic slope at $M = 5H$, and the gap between the two methods at the same iteration budget ($1.5 \cdot 10^{-6}$ for IRLS at iteration 100 versus $4.6 \cdot 10^{-4}$ for SGPTL at iteration 8000) lines up with what the $O(\log \varepsilon^{-1})$ versus $O(\varepsilon^{-2})$ contrast predicts: SGPTL needed roughly 3000× the iteration count IRLS used to reach $\varepsilon = 10^{-2}$, and another factor of 6× to reach 10^{-3}.

Where the theoretical bounds proved conservative. A few observations did not have a clean theoretical counterpart in the report and are worth recording. The IRLS final gap saturated at around 10^{-10} for $\varepsilon_{\mathrm{thr}} = 10^{-12}$, but pushing $\varepsilon_{\mathrm{thr}}$ further did not improve the gap further: the linear solver hits floating-point limits before the safety mechanism does, so the "smaller $\varepsilon_{\mathrm{thr}}$ is always sharper" intuition has a hard floor. The SGPTL contraction factor ρ has a clear effect on the final gap, with a U-shaped sensitivity centred near $\rho = 0.9$ that matches the report's recommendation in Section 3.4.1; the extremes either contract δ too quickly ($\rho = 0.5$, no time for a better f_{ref} to be found before the target collapses on it) or too slowly ($\rho = 0.99$, target stays below f^* for too long). Finally, the OLS warm start mandated by Section 3.4 interacts unfavourably with the greedy deflection. The current subgradient becomes nearly aligned with $\mathbf{d}_{i-1}$ within a few tens of iterations, $\gamma_i \to 0$, and the stepsize-restricted $\beta_i = \min(\beta, \gamma_i)$ multiplies the Polyak step by that same vanishing factor, so the iterate stops moving before the travel-distance counter r can ever reach R to trigger a δ-contraction. We had to add an iteration-count fallback to the patience mechanism — contract δ and reset $\mathbf{d}_{i-1} = \mathbf{0}$ when $\bar{f}^i$ has not improved for $R_{\text{iter}} = \max(i_{\max}/100, 50)$ iterations — to recover the sublinear progress the theory predicts. The reset is consistent with the per-step bound (3.14), which holds iterate-by-iterate without requiring persistent memory.

Recommendation for the ELM LASSO problem. On the project's intended regime, $M \gg H$ with $\mathbf{W}_1$ fixed random and the output layer trained offline once, IRLS is the natural choice. It has a single numerical knob (ε_{thr}), delivers genuinely sparse iterates without post-processing, and reaches machine-precision accuracy in tens to a few hundred iterations on every problem size we tested. SGPTL becomes competitive only when $H^2 \gg M$, i.e. when the hidden layer is wide and the training set is small enough that the $O(MH^2)$ pre-computation cost of IRLS no longer amortises, or when the dense $H \times H$ matrix $\mathbf{X}^\top \mathbf{X}$ does not fit in memory. Neither regime applies to the project's setting; the moderate hidden-layer sizes accessible on a laptop ($H \leq 2000$) all fall on the IRLS side of the crossover. SGPTL retains value as a sanity check (a different algorithm converging to the same f^* within the achievable accuracy is a useful indication that the implementation is correct) and as a baseline to quantify what we gain from exploiting the structure of the LASSO problem through the MM smoothing rather than treating it as a generic non-smooth convex programme.

Limitations and possible extensions. We did not test the algorithms on real ELM datasets because the project's evaluation criterion is the optimisation gap, not the learning quality, and synthetic instances with known $\mathbf{w}^*$ are the only setting where the gap is meaningful. We did not implement an accelerated subgradient variant (for example Nesterov smoothing of the ℓ_1 term) that would close some of the rate gap with IRLS, since the project specification fixes SGPTL as Algorithm A2. We also did not explore very ill-conditioned $\mathbf{X}$ where the IRLS linear rate would degrade and the per-iteration cost advantage of SGPTL would have more room to matter; column normalisation kept conditioning bounded across all our experiments, which is the regime our analysis covers.

Bibliography

[1] Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004.

[2] Sébastien Bubeck. *Convex Optimization: Algorithms and Complexity*. arXiv:1405.4980v2. 2015.

[3] Rick Chartrand and Wotao Yin. "Iteratively reweighted algorithms for compressive sensing". In: *2008 IEEE international conference on acoustics, speech and signal processing*. IEEE. 2008, pp. 3869–3872.

[4] Alice Cortinovis. *Introduction to Least Squares Problem*. Lecture notes: Intro to Least Squares, CM 646AA, Università di Pisa. 2025.

[5] Gianluca d'Antonio and Antonio Frangioni. "Convergence Analysis of Deflected Conditional Approximate Subgradient Methods". In: *SIAM Journal on Optimization* 20.1 (2009), pp. 357–386.

[6] Ingrid Daubechies et al. *Iteratively re-weighted least squares minimization for sparse recovery*. 2008. arXiv: `0807.0575 [math.NA]`. URL: `https://arxiv.org/abs/0807.0575`.

[7] Antonio Frangioni. *Convergence analysis of Diminishing-Square Summable (DSS) stepsize*. `https://elearning.di.unipi.it/pluginfile.php/90522/mod_resource/content/5/5-nonsmooth%20unconstrained%20optimization.pdf`. Slide 11. 2025.

[8] Antonio Frangioni. *Convex Functions*. Lecture notes: Unconstrained Multivariate Optimality and Convexity, CM 646AA, Università di Pisa `https://elearning.di.unipi.it/pluginfile.php/90520/mod_resource/content/8/3-unconstrained%20optimality.pdf`. Slides 21–24. 2025.

[9] Antonio Frangioni. *Convex nondifferentiable optimization is harder: lower bounds*. `https://elearning.di.unipi.it/pluginfile.php/90522/mod_resource/content/5/5-nonsmooth%20unconstrained%20optimization.pdf`. Slide 8. 2025.

[10] Antonio Frangioni. *Deflected subgradient*. `https://elearning.di.unipi.it/pluginfile.php/90522/mod_resource/content/5/5-nonsmooth%20unconstrained%20optimization.pdf`. University of Pisa, Slide 15. 2026.

[11] Antonio Frangioni. *Introduction to subgradients and subdifferentials*. Lecture notes: Nonsmooth Unconstrained Optimization, CM 646AA, Università di Pisa. Slides 5–6. 2025.

[12] Antonio Frangioni. *Lipschitz continuity, L-smoothness*. Lecture notes: Smooth Unconstrained Multivariate Optimization, CM 646AA, Università di Pisa. Slide 9. 2025.

[13] Antonio Frangioni. *Nondifferentiable regularization: L1 as best convex approximation of L0*. Lecture notes: Nonsmooth Unconstrained Optimization, CM 646AA, Università di Pisa. Slide 3. 2025.

[14] Antonio Frangioni. *Polyak stepsize*. `https://elearning.di.unipi.it/pluginfile.php/90522/mod_resource/content/5/5-nonsmooth%20unconstrained%20optimization.pdf`. University of Pisa, Slide 12. 2026.

[15] Antonio Frangioni. *Smooth methods fail on nonsmooth functions: nondifferentiability of L1 norm*. Lecture notes: Nonsmooth Unconstrained Optimization, CM 646AA, Università di Pisa. Slide 4. 2025.

[16] Antonio Frangioni. *Stronger forms of convexity (strongly convex, strictly convex)*. Lecture notes: Smooth Unconstrained Multivariate Optimization, CM 646AA, Università di Pisa. Slide 12. 2025.

[17] Antonio Frangioni. *Subdifferential calculus: sum rule for subdifferentials*. `https://elearning.di.unipi.it/pluginfile.php/90522/mod_resource/content/5/5-nonsmooth%20unconstrained%20optimization.pdf`. University of Pisa, Slide 7. 2026.

[18] Antonio Frangioni. *Subgradient methods: fundamental relationships*. `https://elearning.di.unipi.it/pluginfile.php/90522/mod_resource/content/5/5-nonsmooth%20unconstrained%20optimization.pdf`. University of Pisa, Slide 10. 2026.

[19] Antonio Frangioni. *Target level stepsize (vanishing)*. `https://elearning.di.unipi.it/pluginfile.php/90522/mod_resource/content/5/5-nonsmooth%20unconstrained%20optimization.pdf`. University of Pisa, Slide 14. 2026.

[20] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. URL: `https://www.deeplearningbook.org`.

[21] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning*. 2nd. Springer, 2009.

[22] Guang-Bin Huang, Qin-Yu Zhu, and Chee-Kheong Siew. "Extreme learning machine: Theory and applications". In: *Neurocomputing* 70.1 (2006), pp. 489–501. DOI: `10.1016/j.neucom.2005.12.126`.

[23] Hien D. Nguyen. *An Introduction to MM Algorithms for Machine Learning and Statistical*. 2016. arXiv: `1611.03969 [stat.CO]`. URL: `https://arxiv.org/abs/1611.03969`.

[24] Ali Rahimi and Benjamin Recht. “Random Features for Large-Scale Kernel Machines”. In: *Advances in Neural Information Processing Systems*. Vol. 20. 2007.

[25] Lloyd N. Trefethen and David Bau. *Numerical Linear Algebra*. SIAM, 1997.